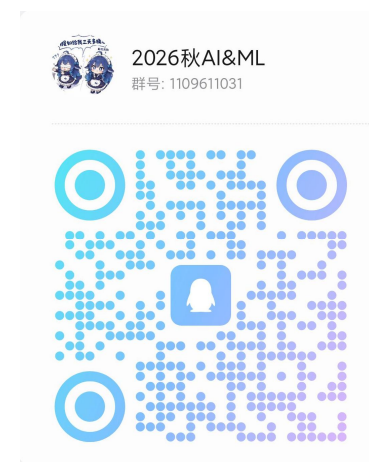


人工智能与机器学习基础

Lecture 04: Regression & Regularization

冯文杰

人工智能与数据科学学院
中国科学技术大学



提纲

- 模型拟合
- 岭回归模型
- **Lasso** 回归模型
- 神经网络正则化

提纲

- **模型拟合与修正**

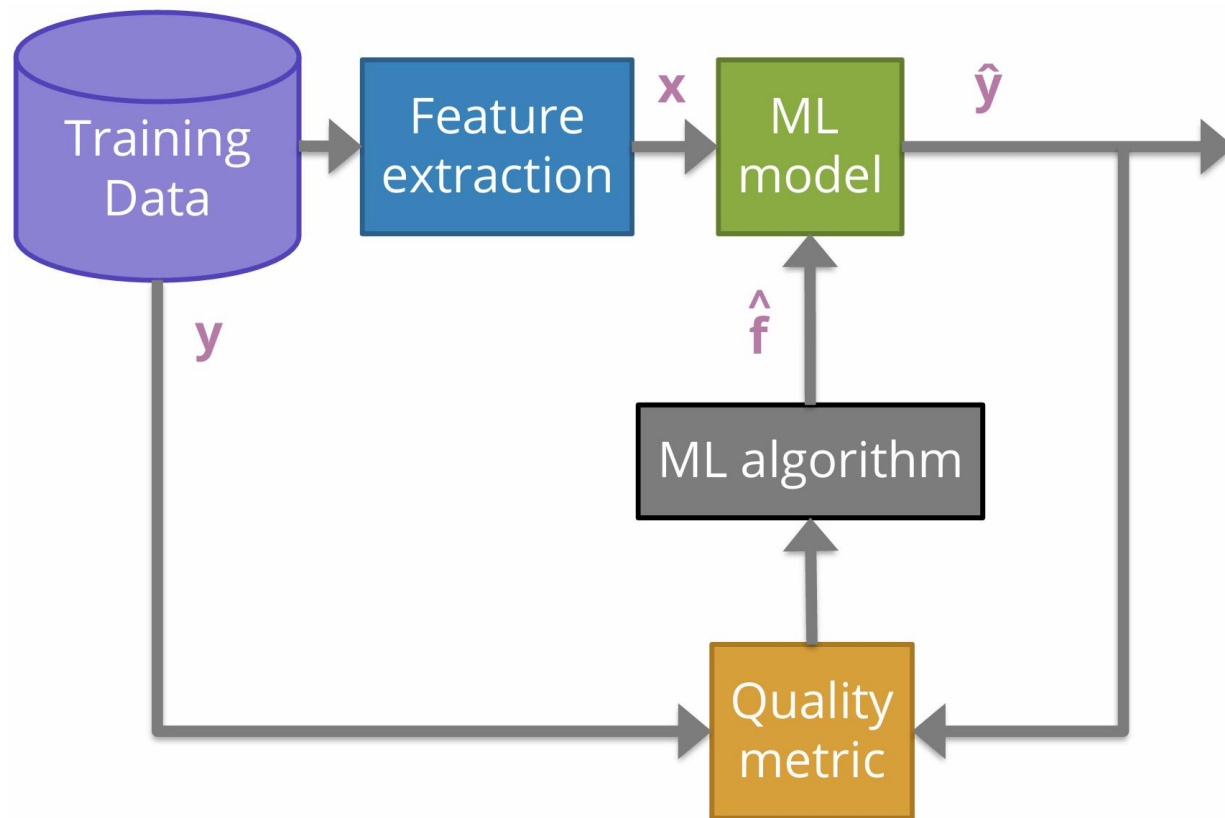
- 过拟合
- 正则项

- 岭回归模型

- **Lasso** 回归模型

- 神经网络正则化

机器学习流程

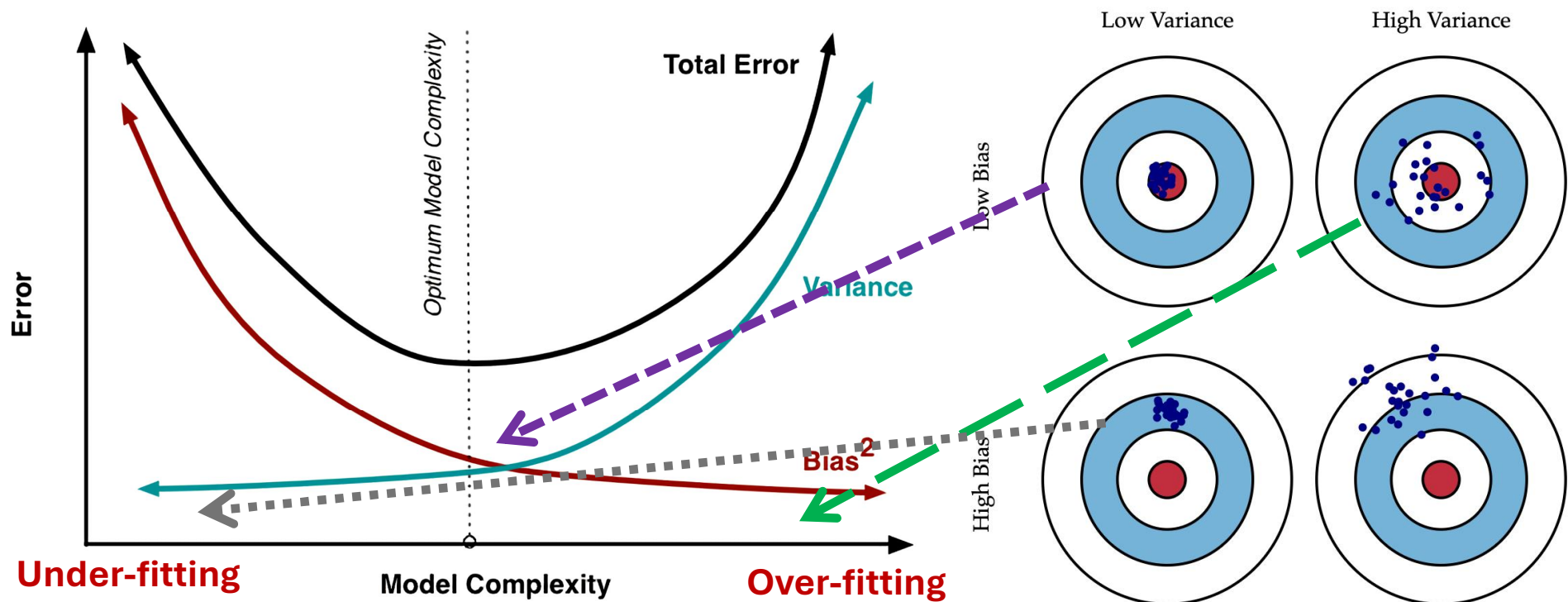


效果好 or 过拟合?



Recap: 偏差-方差权衡

- 偏差-方差均衡



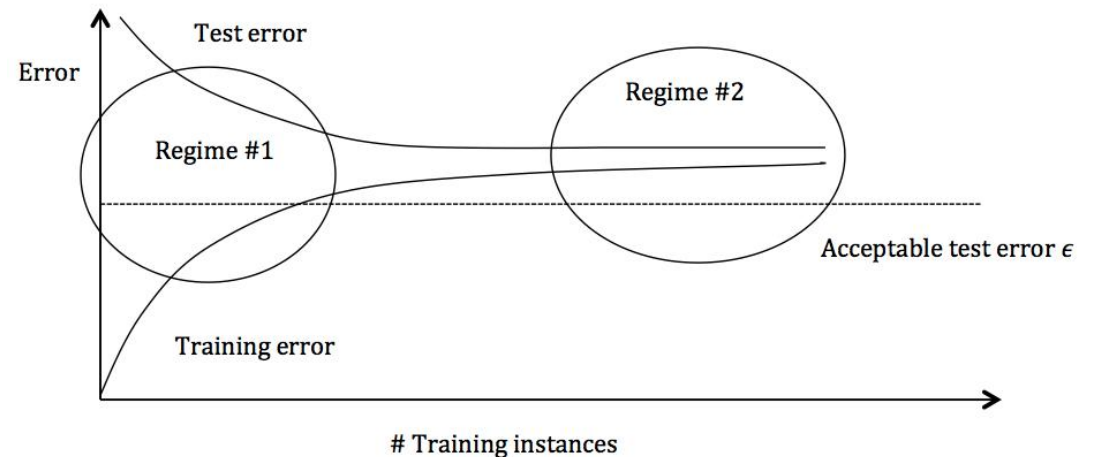
Recap: 高方差和高偏差检测

- **Regime 1**

- High Variance
- Symptoms
 - Training error $\ll$ Test error
 - Training error $< \epsilon$
 - Test error $> \epsilon$

- Remedies

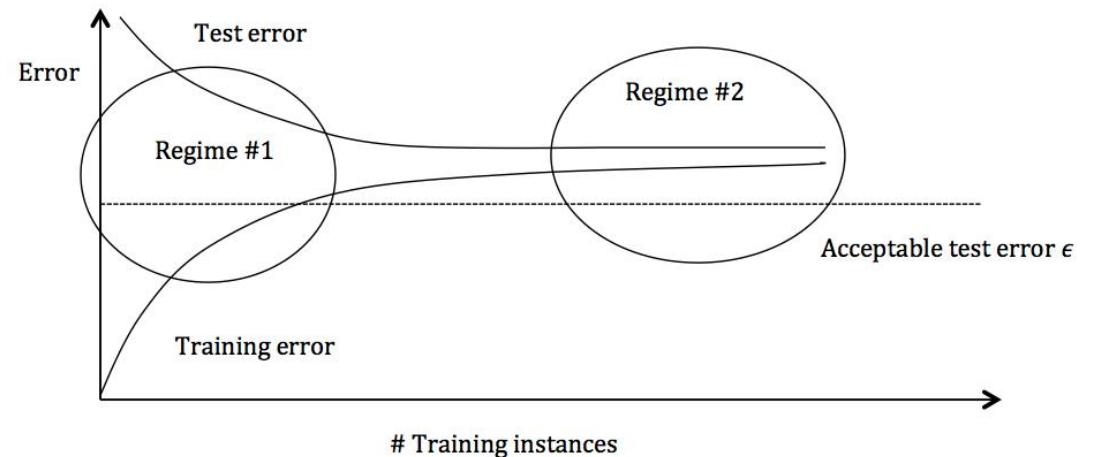
- Add more training data
- Reduce model complexity
 - Complex models are prone to high variance
- Bagging



Recap: 高方差和高偏差检测

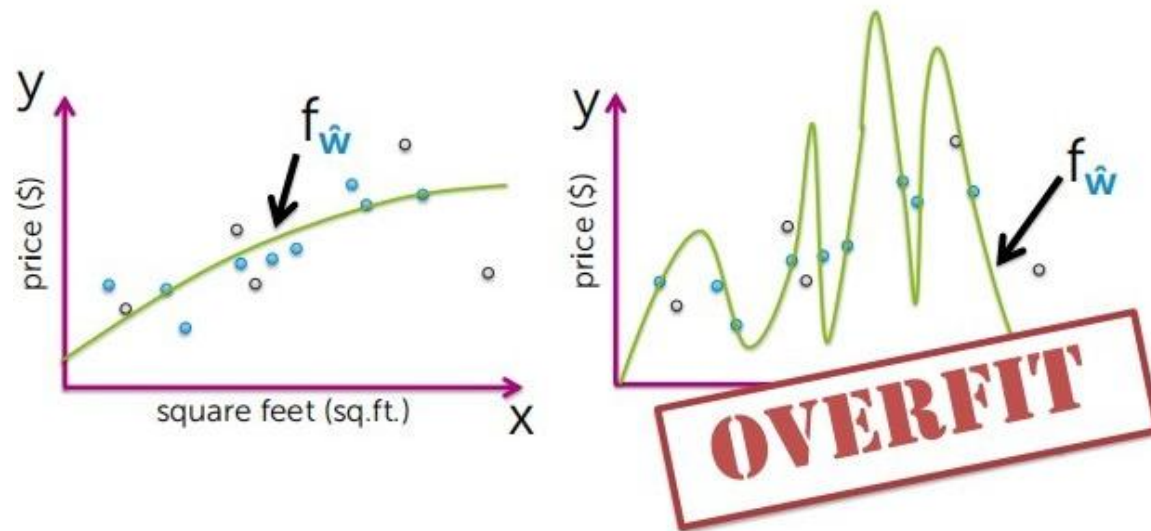
- **Regime 2**

- High Bias
- Symptoms
 - Training error $> \epsilon$
- Remedies
 - Add features
 - Use more complex model
 - kernelize, use non-linear models
 - Boosting



高阶多项式的灵活性

$$y_i = w_0 + w_1 x_i + w_2 x_i^2 + \dots + w_p x_i^p + \varepsilon_i$$

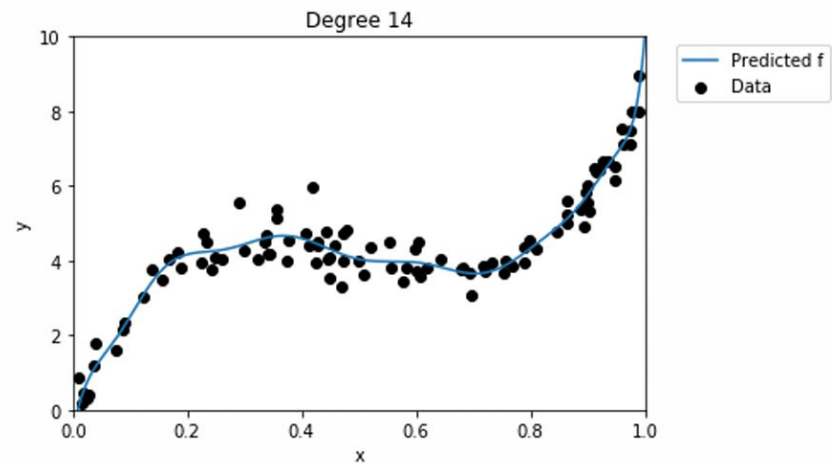
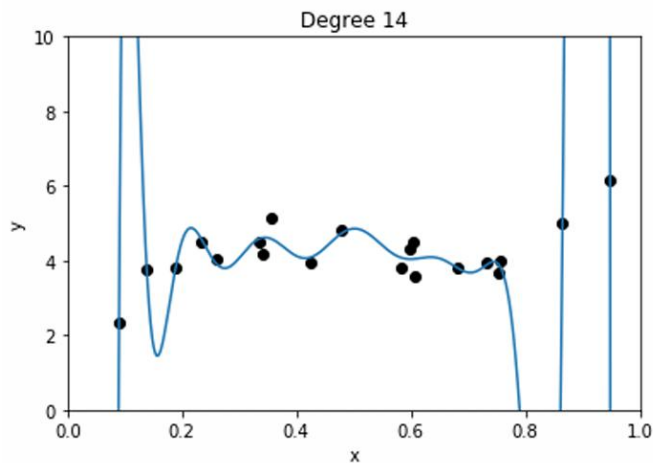


高阶多项式具有很强表达能力，可通过增加曲折和弯曲来紧密贴合训练数据。

- **优点：**能够拟合复杂的非线性关系，训练误差通常较小。
- **缺点：**灵活性过高也会导致方差过大，模型在不同训练集上表现差异显著，从而容易产生过拟合。

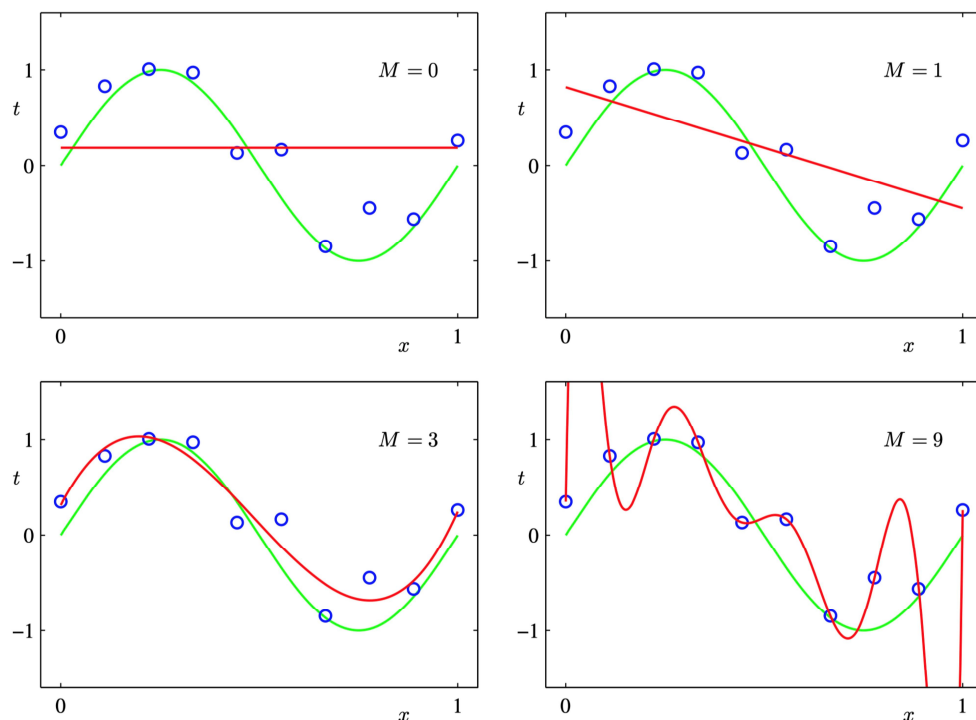
特征数

- **过拟合**并不局限于高次多项式回归
 - 当你使用大量特征时，同样可能出现过拟合！
- 原因是什么？
 - 过拟合取决于你所拥有的**数据量**，以及这些数据是否足以为模型的复杂性提供具有**代表性**的样本。



过拟合征兆

- 多项式拟合的过拟合问题



	$M = 0$	$M = 1$	$M = 6$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

过拟合的征兆

- 过拟合往往与**非常大的估计参数 $\hat{w}$** 有关。
- **现象**
 - 大的 $\hat{w}$ 使模型函数在区间内剧烈震荡，训练误差虽小，但泛化误差大。
- **结论**
 - 参数过大可以视作过拟合的信号之一。
- 在损失函数中增加一个额外的项，可以鼓励模型在优化过程中选择**较小的参数系数**。

提纲

- **模型拟合与修正**
 - 过拟合
 - 正则项
- 岭回归模型
- **Lasso** 回归模型
- 神经网络正则化

解决方法

在损失函数中增加一个额外的项，可以鼓励模型在优化过程中选择较小的参数系数。

期望的总损失形式

- 模型优化的考量：训练误差与泛化性能间的平衡
 - 函数对数据的拟合程度
 - 系数的大小（参数规模）

Total cost = measure of fit + measure of magnitude of coefficients



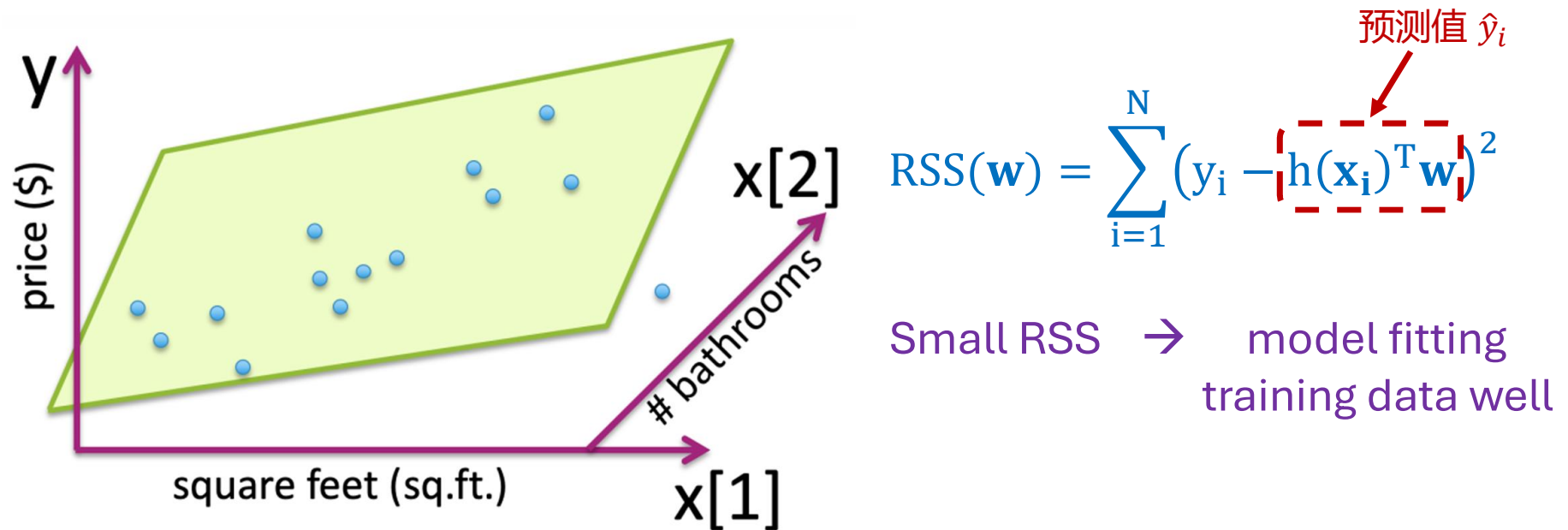
训练误差小 =
说明模型对训练数据拟合得好



参数规模小 =
表明模型没有为了拟合噪声而
膨胀系数，因此不容易过拟合

对训练数据拟合程度的度量

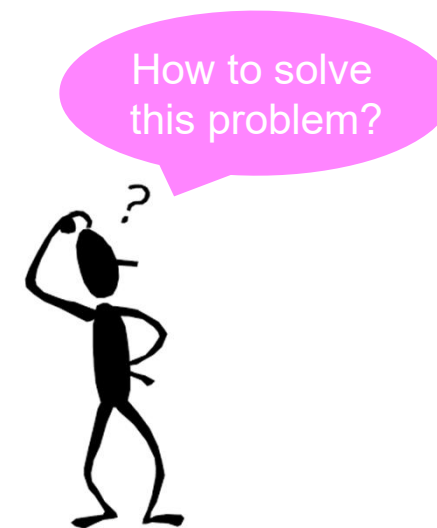
- 残差平方和 (Residual Sum of Squares, RSS)



回归系数大小的度量

- 哪个汇总数值能表征回归系数的大小?

	$M = 0$	$M = 1$	$M = 6$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43



回归系数大小的度量

- 哪个汇总数值能表征回归系数的大小?
 - **简单求和 (Sum)** : 不合适, 因为正负系数会相互抵消。
 - **绝对值和 (Sum of absolute values, L1 norm)** : 常用于 Lasso, 能度量系数整体规模, 并鼓励稀疏性。

$$||\mathbf{w}||_1 = \sum_{j=0}^D |w_j|$$

- **平方和 (Sum of squares, L2 norm)** : 常用于Ridge, 更强调惩罚大系数, 让参数分布更平滑。

$$||\mathbf{w}||_2^2 = \sum_{j=0}^D w_j^2$$

考虑一个具体的总体损失

$$\text{Total cost} = \underbrace{\text{measure of fit}}_{\text{RSS}(\mathbf{w})} + \underbrace{\text{measure of magnitude of coefficients}}_{\|\mathbf{w}\|_2^2}$$

考虑最终的目标函数

如果 $\hat{\mathbf{w}}$ 是通过最小化以下函数得到: $\text{RSS}(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2$

λ : 调节“拟合数据”和“保持系数小”之间的权衡。

- 如果 $\lambda=0$:

退化为普通最小二乘惩罚, 解为 $\hat{\mathbf{w}}^{\text{LS}}$

- 如果 $\lambda=\infty$:

当 $\mathbf{w} \neq \mathbf{0}$, 代价都是 ∞ ; 只有当 $\mathbf{w} = \mathbf{0}$ 时, 代价 = $\text{RSS}(\mathbf{0})$

- 如果 λ 取中间值:

解介于两者之间, 满足 $0 \leq \|\hat{\mathbf{w}}\|_2^2 \leq \|\hat{\mathbf{w}}^{\text{LS}}\|_2^2$

考虑最终的目标函数

- 如果 $\hat{\mathbf{w}}$ 是通过最小化以下函数得到：

$$\text{RSS}(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2$$

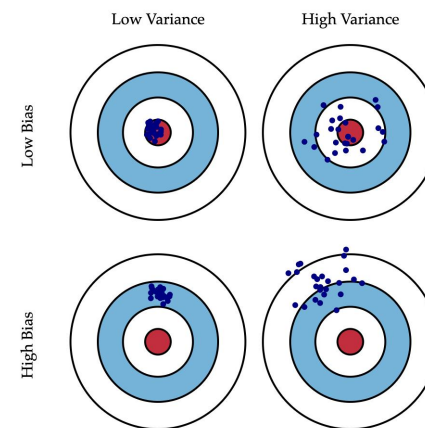

λ : 调节“拟合数据”和“保持系数小”之间的权衡。

**Ridge regression, 岭回归
(a.k.a. L2 regularization, L2正则化)**

偏差-方差权衡

- Large λ
 - **偏差高 (high bias)** , **方差低 (low variance)**
 - 例如: 当 $\lambda = \infty$ 时, $\hat{\mathbf{w}} = \mathbf{0}$, 模型退化为常数预测
- Small λ
 - **偏差低 (low bias)** , **方差高 (high variance)**
 - 例如: $\lambda = 0$ 时, 就是高阶多项式的最小二乘拟合, 容易过拟合

本质上, λ 控制模型的复杂度

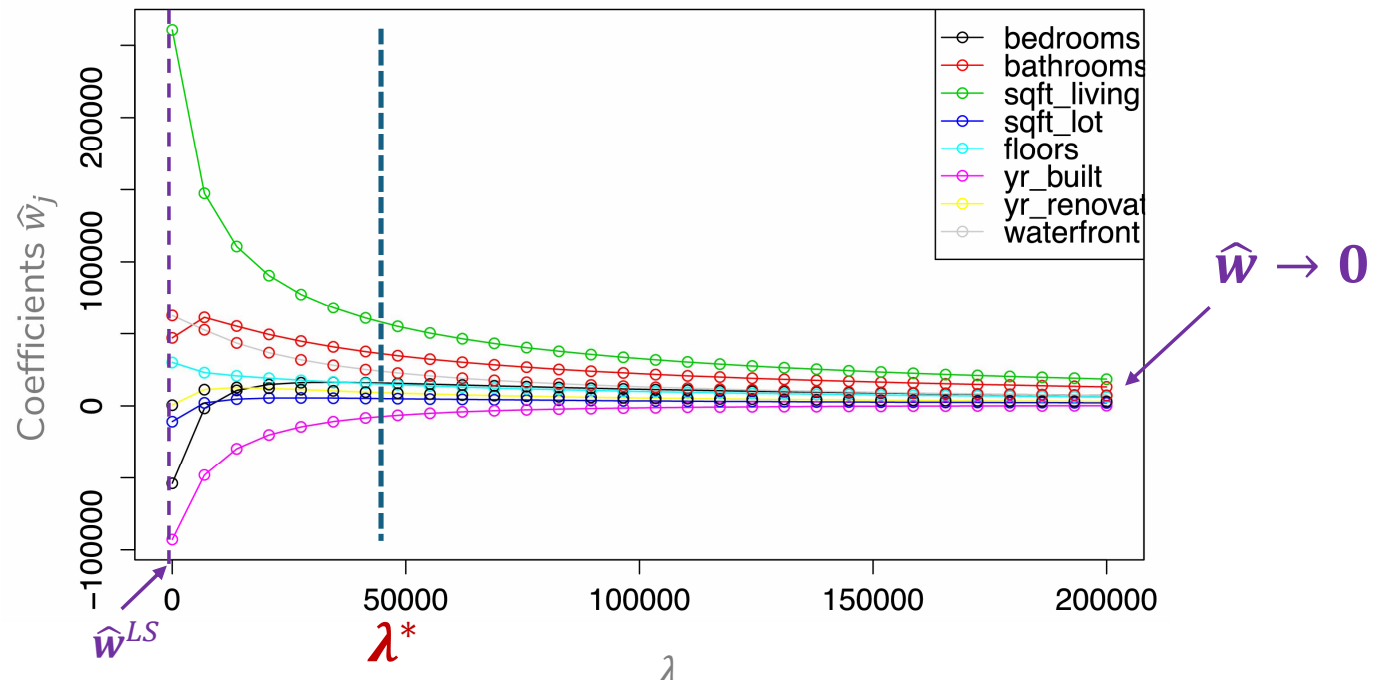


重看多项式拟合示例

- 如果我们重新拟合同一个高阶多项式，但这次使用岭回归，会发生什么？
- 我们将考虑几个不同的 λ 设置.....

系数路径

- **系数路径**是指在正则化回归（如 Ridge或 Lasso）中，随着正则化参数 λ 变化， $\hat{\mathbf{w}}$ 中各个回归系数的取值轨迹。
- **岭回归(L2)**：系数随 λ 增大逐渐向 0 收缩，但通常不会完全为 0。



提纲

- 模型拟合与修正
- **岭回归模型**
 - 模型拟合
 - 正则超参选择
- **Lasso 回归模型**
- 网络正则化

提纲

- 模型拟合与修正
- **岭回归模型**
 - 模型拟合
 - 正则超参选择
- **Lasso** 回归模型
- 神经网络正则化

岭回归代价函数的梯度

$$\begin{aligned}\nabla \left[\text{RSS}(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2 \right] &= \nabla [(\mathbf{y} - \mathbf{H}\mathbf{w})^T (\mathbf{y} - \mathbf{H}\mathbf{w}) + \lambda \mathbf{w}^T \mathbf{w}] \\ &= \underbrace{\nabla [(\mathbf{y} - \mathbf{H}\mathbf{w})^T (\mathbf{y} - \mathbf{H}\mathbf{w})]}_{-2\mathbf{H}^T (\mathbf{y} - \mathbf{H}\mathbf{w})} + \lambda \underbrace{\nabla [\mathbf{w}^T \mathbf{w}]}_{2\mathbf{w}}\end{aligned}$$

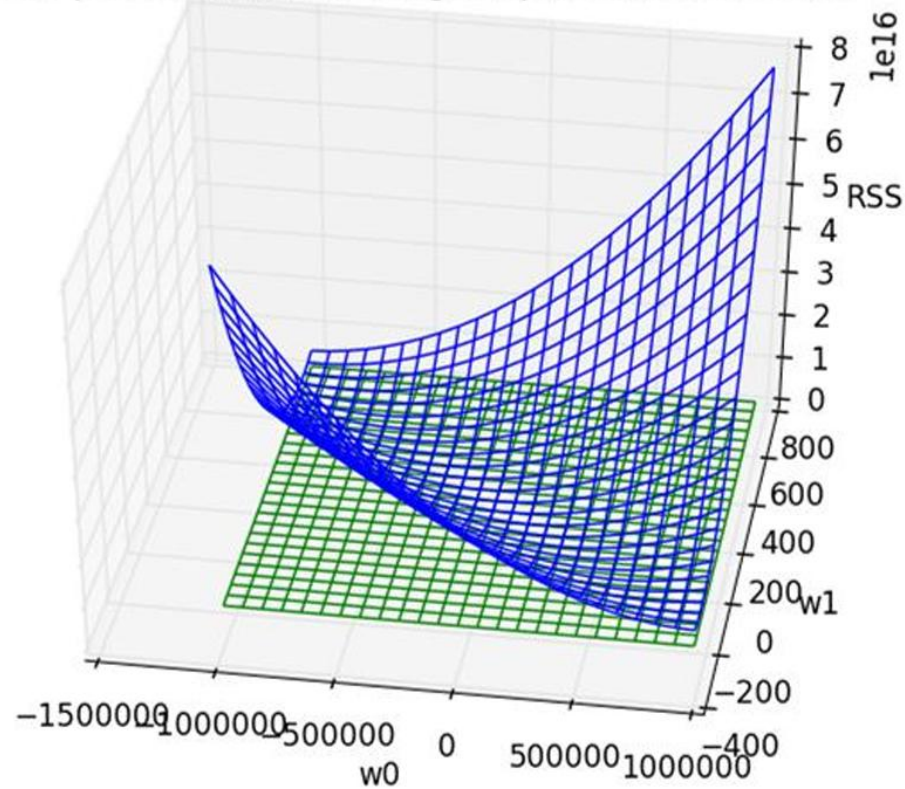
$$\begin{aligned}\|\mathbf{w}\|_2^2 &= w_0^2 + w_1^2 + \cdots + w_D^2 \\ &= \mathbf{w}^T \mathbf{w} \\ &= [w_0, \cdots, w_D][w_0, \cdots, w_D]^T\end{aligned}$$

• 如何求梯度？ 类比一维情况...

• $\mathbf{w}^T \mathbf{w}$ 类似于 w^2 , $\frac{\partial w^2}{\partial w} = 2w$

岭回归的闭式解

3D plot of RSS with tangent plane at minimum



$$\begin{aligned}\nabla \text{cost}(\mathbf{w}) &= -2\mathbf{H}^T(\mathbf{y} - \mathbf{H}\mathbf{w}) + 2\lambda\mathbf{I}\mathbf{w} \\ &= \mathbf{0}\end{aligned}$$

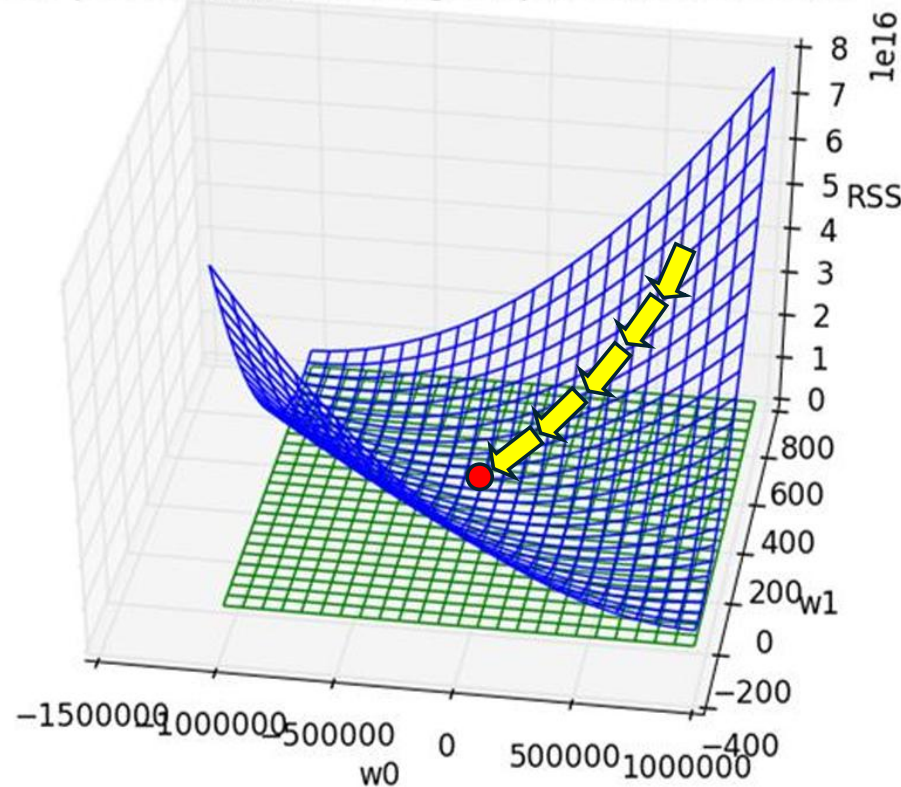
求解 $\mathbf{w}$: “Regularizer”

$$\hat{\mathbf{w}}^{\text{ridge}} = (\mathbf{H}^T\mathbf{H} + \lambda\mathbf{I})^{-1}\mathbf{H}^T\mathbf{y}$$

岭回归的梯度下降

- $\nabla \text{cost}(\mathbf{w}) = -2\mathbf{H}^T(\mathbf{y} - \mathbf{H}\mathbf{w}) + 2\lambda\mathbf{w}$

3D plot of RSS with tangent plane at minimum



Algorithm

While not converged:

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \underbrace{\eta \nabla \text{cost}(\mathbf{w}^{(t)})}_{-2\mathbf{H}^T(\mathbf{y} - \mathbf{H}\mathbf{w}^{(t)}) + 2\lambda\mathbf{w}^{(t)}}$$

提纲

- 模型拟合与修正
- **岭回归模型**
 - 模型拟合
 - 正则超参选择
- **Lasso** 回归模型
- 神经网络正则化

回归/机器学习工作流程

1. 模型选择 (Model selection)
 - 需要选择控制模型复杂度的超参数 λ
2. 模型评估 (Model assessment)
 - 在选定模型之后, 需要评估其泛化误差 (generalization error)

假想的实现流程

Training set

Test set

Overly
Optimistic!

过于乐观

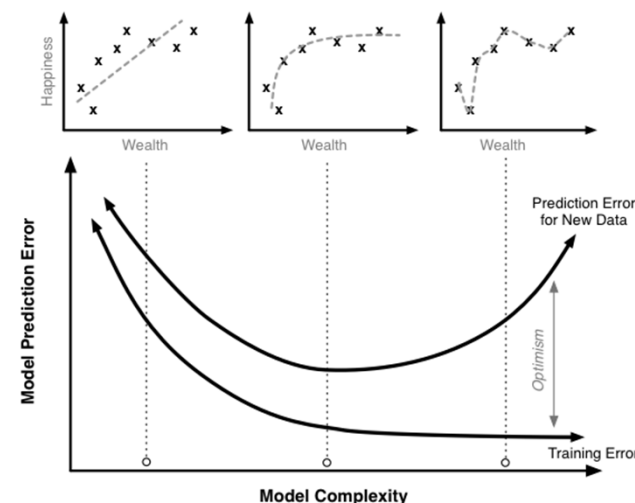
1. 模型选择(Model selection)

对每个候选的 λ :

- 在训练数据上估计参数 $\hat{\mathbf{w}}_\lambda$
- 在测试数据上评估的 $\hat{\mathbf{w}}_\lambda$ 性能
- 选择测试误差最低的 λ^*

2. 模型评估(Model assessment)

- 计算 $\hat{\mathbf{w}}_{\lambda^*}$ (即选定 λ^* 后得到的模型) 的测试误差
- 用以近似泛化误差 (generalization error)



假设的实现流程



问题在于：就像在训练数据上同时拟合 $\hat{\mathbf{w}}$ 并评估其性能一样

- λ^* 是通过最小化测试误差选出来的
 - 也就是说, λ^* 是在测试集上 “拟合” 的。
- 如果测试集并不能代表真实世界的分布, 那么 $\hat{\mathbf{w}}_{\lambda^*}$ 的实际表现通常会比测试误差所显示的更差。

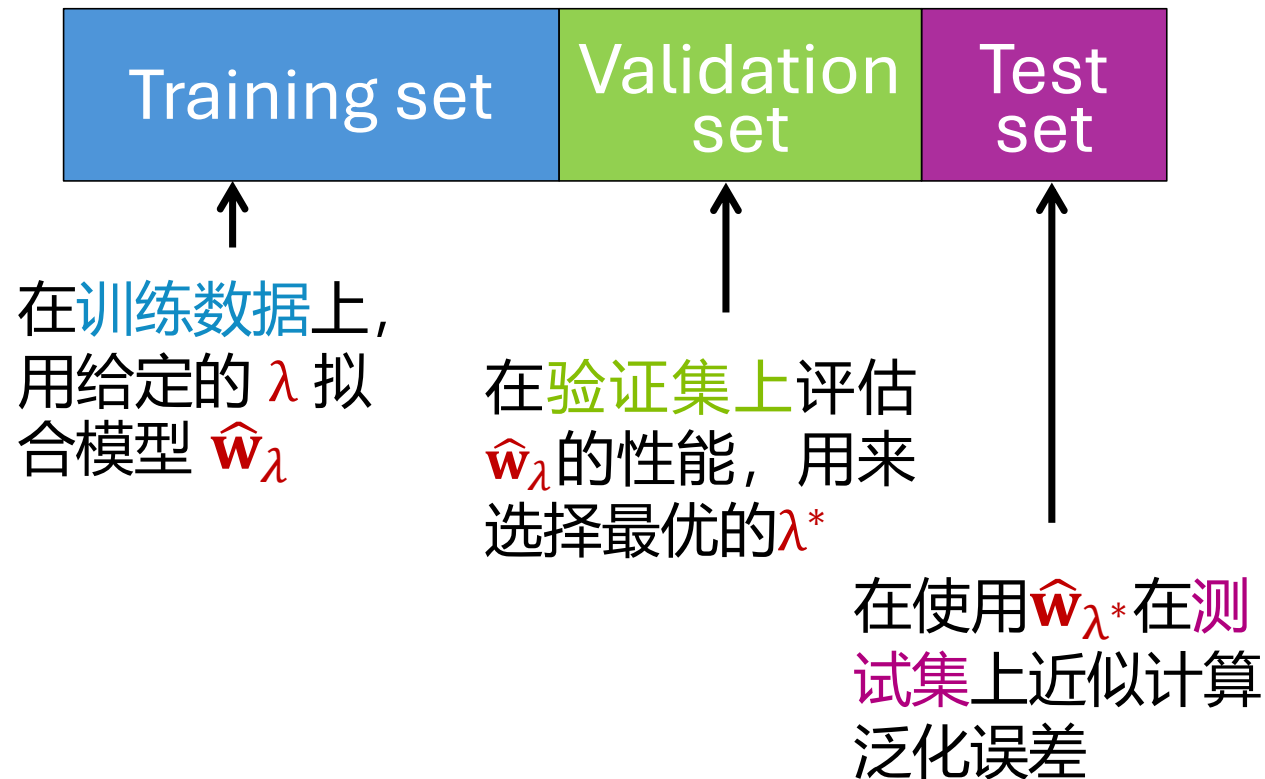
实际的实现流程



解决方案：创建两个“测试”集！

1. **使用验证集(validation set)：**选择使得 $\hat{\mathbf{w}}_{\lambda^*}$ 在验证集上误差最小的 λ^* 。
2. **使用测试集(test set)：**通过测试集来近似评估 $\hat{\mathbf{w}}_{\lambda^*}$ 的泛化误差。

实际的实现流程



常用划分比例



80%

10%

10%

50%

25%

25%

提纲

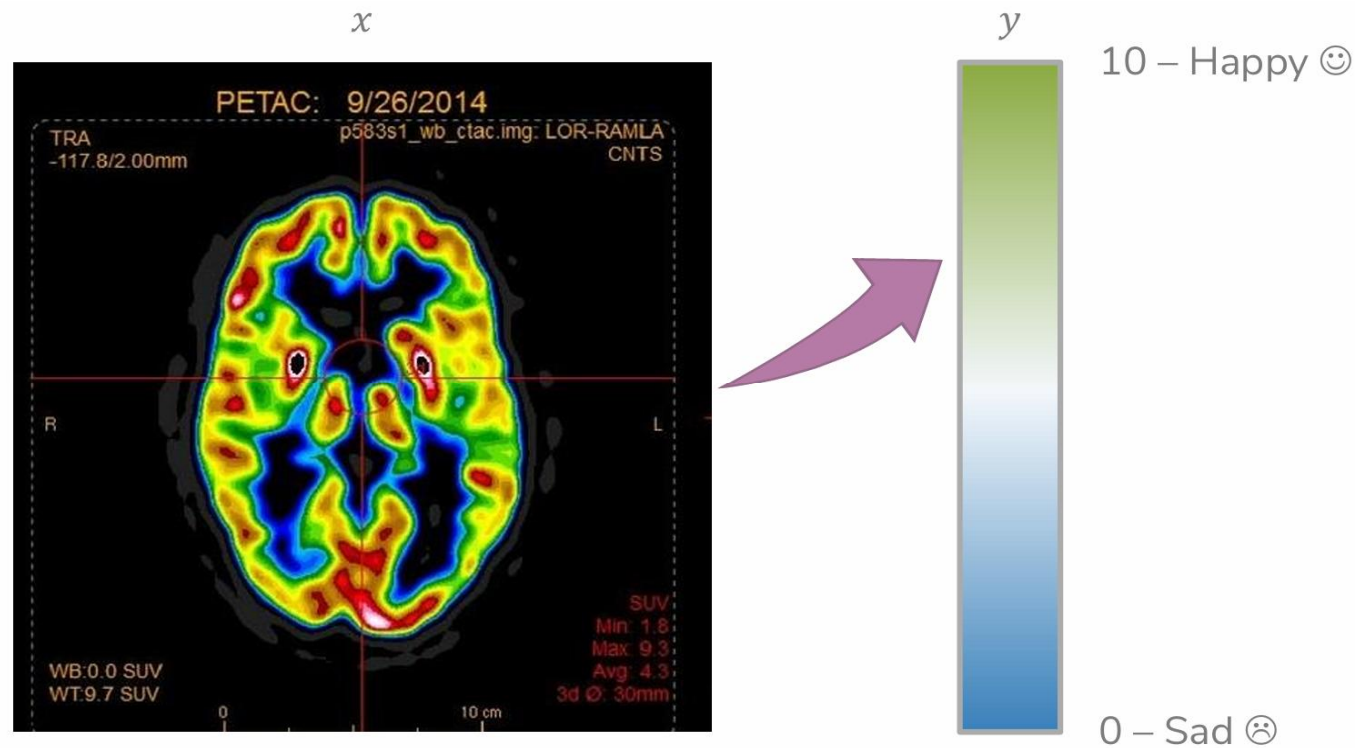
- 模型拟合与修正
- 岭回归模型
- **Lasso 回归模型**
 - 特征选择任务
 - 模型拟合
 - 正则超参选择
 - 解稀疏性几何直观
- 网络正则化

提纲

- 模型拟合与修正
- 岭回归模型
- **Lasso 回归模型**
 - 特征选择任务
 - 模型拟合
 - 正则超参选择
 - 解稀疏性几何直观
- 神经网络正则化

稀疏性: Reading Minds

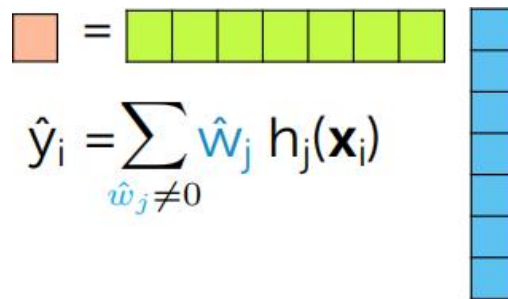
- 你有多快乐?
- 大脑的哪个部位负责控制快乐感?



为什么要进行特征选择?

效率 (Efficiency) :

- 如果 $|\mathbf{w}|_0 = 100B$, 那么每一次预测的计算开销都很大。
- 如果 $\hat{\mathbf{w}}$ 是稀疏的, 计算量只取决于非零系数的数量。


$$\hat{y}_i = \sum_{\hat{w}_j \neq 0} \hat{w}_j h_j(\mathbf{x}_i)$$

可解释性 (Interpretability) :

- 哪些特征真正与预测相关?
- 通过特征选择, 可以明确模型依赖的关键变量。

考虑买房这个场景



Lot size
Single Family
Year built
Last sold price
Last sale price/sqft
Finished sqft
Unfinished sqft
Finished basement sqft
floors
Flooring types
Parking type
Parking amount
Cooling
Heating
Exterior materials
Roof type
Structure style

Dishwasher
Garbage disposal
Microwave
Range / Oven
Refrigerator
Washer
Dryer
Laundry location
Heating type
Jetted Tub
Deck
Fenced Yard
Lawn
Garden
Sprinkler System

...

提纲

- 模型拟合与修正
- 岭回归模型
- Lasso 回归模型
 - 特征选择任务：1. 遍历 or 贪心； 2. 正则化
 - 模型拟合
 - 正则超参选择
 - 解稀疏性几何直观
- 神经网络正则化

遍历所有特征子集

- 枚举所有可能的特征子集，选择使模型表现最佳的那个。
 - 优点：可以找到最优解。
 - 缺点：计算复杂度极高（特征数目多时不可行）。

$$\hat{y}_i = \epsilon_i$$

$$\hat{y}_i = w_0 h_0(x) + \epsilon_i$$

$$\hat{y}_i = w_1 h_1(x) + \epsilon_i$$

...

$$\hat{y}_i = w_0 h_0(x) + w_1 h_1(x) + \epsilon_i$$

...

$$\hat{y}_i = w_0 h_0(x) + w_1 h_1(x) + \dots + w_D h_D(x) + \epsilon_i$$

$$[0 \ 0 \ 0 \ \dots \ 0 \ 0 \ 0]$$

$$[1 \ 0 \ 0 \ \dots \ 0 \ 0 \ 0]$$

$$[0 \ 1 \ 0 \ \dots \ 0 \ 0 \ 0]$$

...

$$[1 \ 1 \ 0 \ \dots \ 0 \ 0 \ 0]$$

...

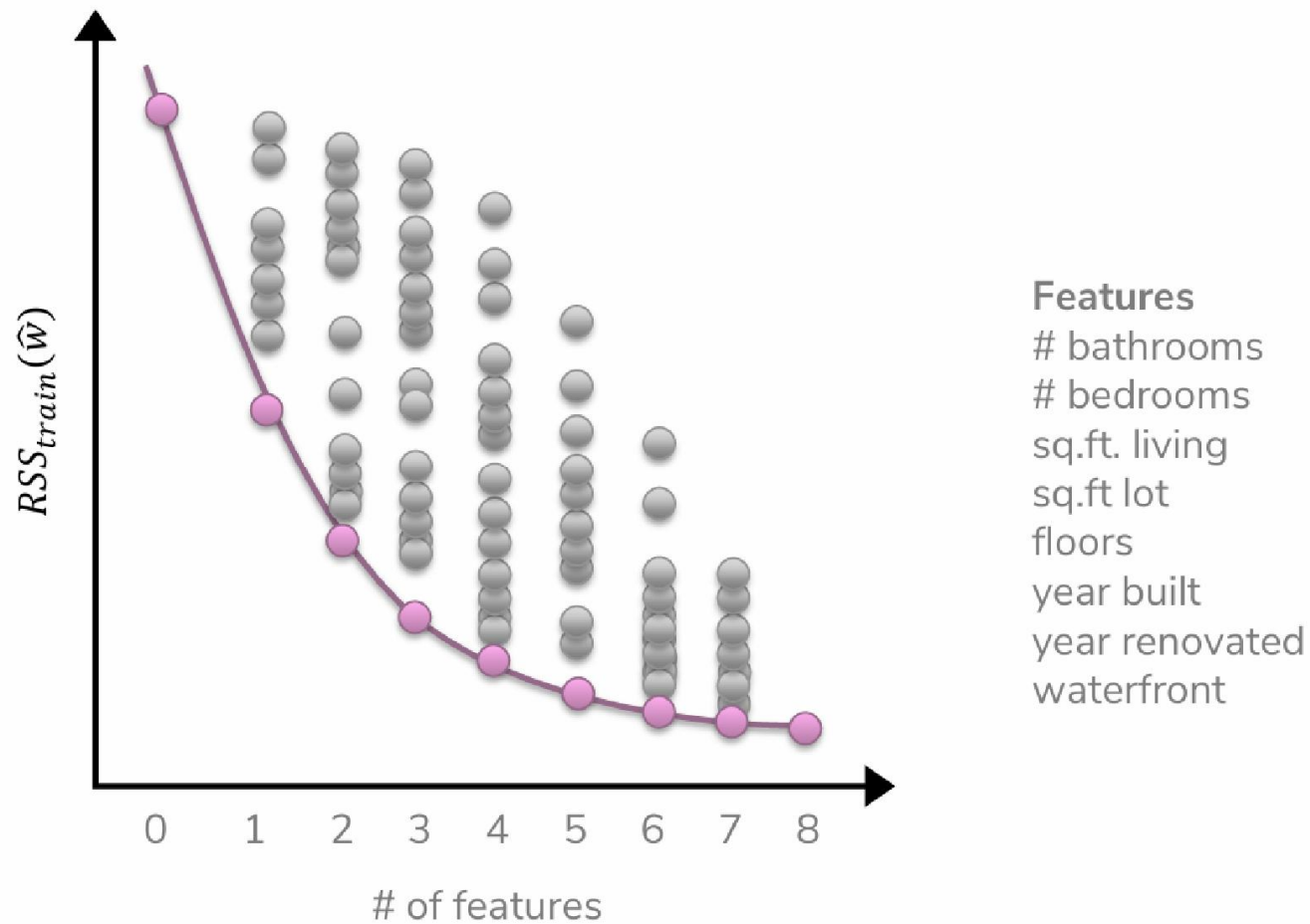
$$[1 \ 1 \ 1 \ \dots \ 1 \ 1 \ 1]$$

$$2^{D+1}$$

如果评估 8 个特征的所有子集
只需要5秒，那么：

- 16个特征需要 21 分钟
- 32个特征需要接近 3 年
- 100个特征需要 $7.5 * 10^{20}$ 年
 - 比宇宙的年龄还要长 500 亿倍！

遍历-Example



贪心算法

- 贪心变体 (Greedy variants)
 - 前向选择 (Forward selection) : 从空集开始, 每次加入一个最优特征。
 - 后向消除 (Backward elimination) : 从全特征开始, 每次去掉一个最不重要的特征。
- 优点: 计算可行, 比穷举高效。
- 缺点: 可能陷入局部最优, 不保证全局最优解。

贪心-Example

```
 $S_0 \leftarrow \{\}$ 
```

```
for  $i \leftarrow 1..k$ :
```

```
    Find feature  $f_i$  not in  $S_{i-1}$ , that when combined  
    with  $S_{i-1}$ , minimizes the loss the most.
```

```
 $S_i \leftarrow S_{i-1} \cup \{f_i\}$ 
```

```
Return  $S_k$ 
```

提纲

- 模型拟合与修正
- 岭回归模型
- Lasso 回归模型
 - 特征选择任务：1. 遍历 or 贪心；2. 正则
 - 模型拟合
 - 正则超参选择
 - 解稀疏性几何直观
- 神经网络正则化

利用正则化进行特征选择

- 不必在离散的解空间搜索子集，我们能否直接通过正则化实现？
- 从完整模型开始（包含所有可能特征）。
- 通过正则化，把部分系数“收缩”到精确为0。
 - 也就是说，某些特征会被自动“剔除”。
- 非零系数所对应的特征就是被“选择”的特征。

**L1 正则化（Lasso 回归）的核心思想：
自动化的特征选择**

回顾：岭回归/L2正则化回归

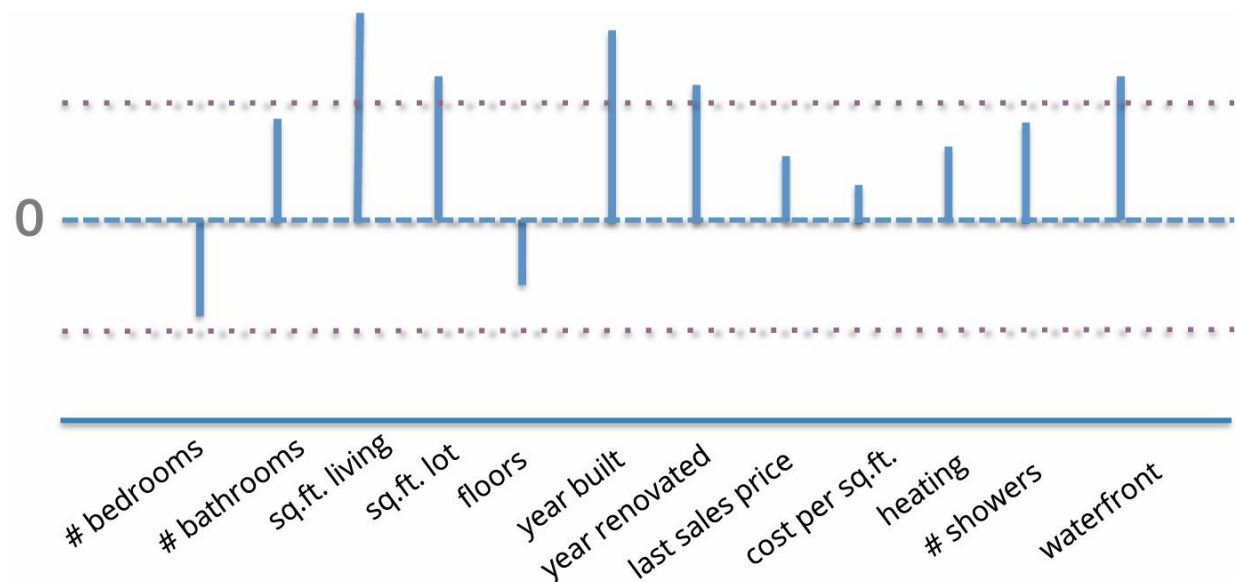
$$\text{Total cost} = \underbrace{\text{measure of fit}}_{\text{RSS}(\mathbf{w})} + \underbrace{\text{measure of magnitude of coefficients}}_{\|\mathbf{w}\|_2^2 = w_0^2 + \dots + w_D^2}$$

鼓励权重变小，
但不会精确收缩到0；

因此，岭回归适合降低方差、防止过拟合，但
不能自动实现特征选择

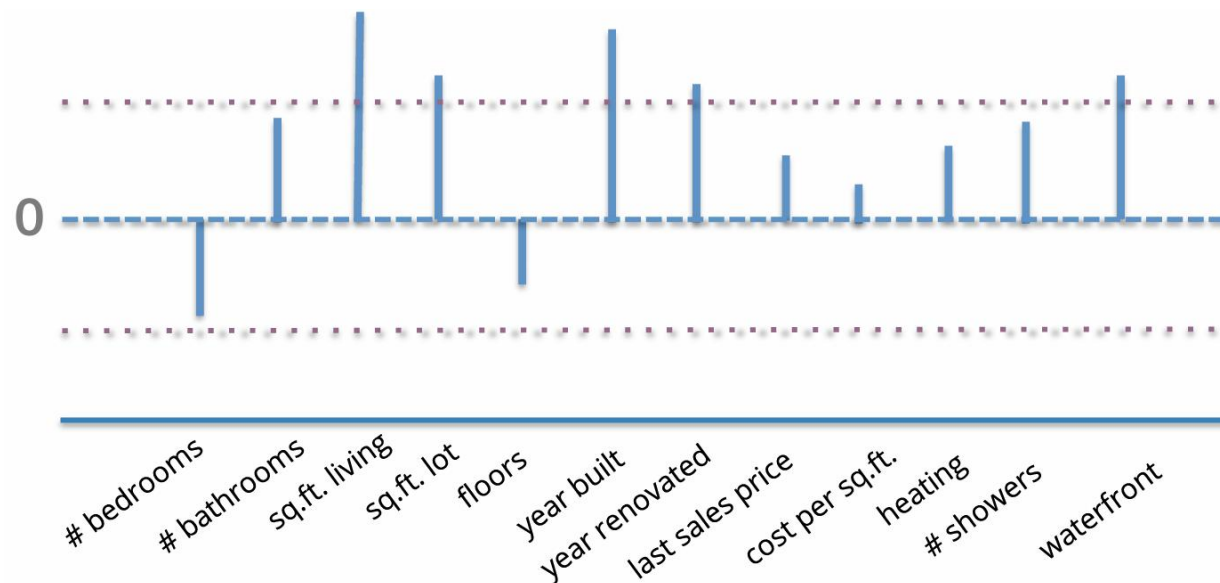
阈值化岭回归系数?

- 为什么我们不能直接把很小的岭回归系数设为 0?



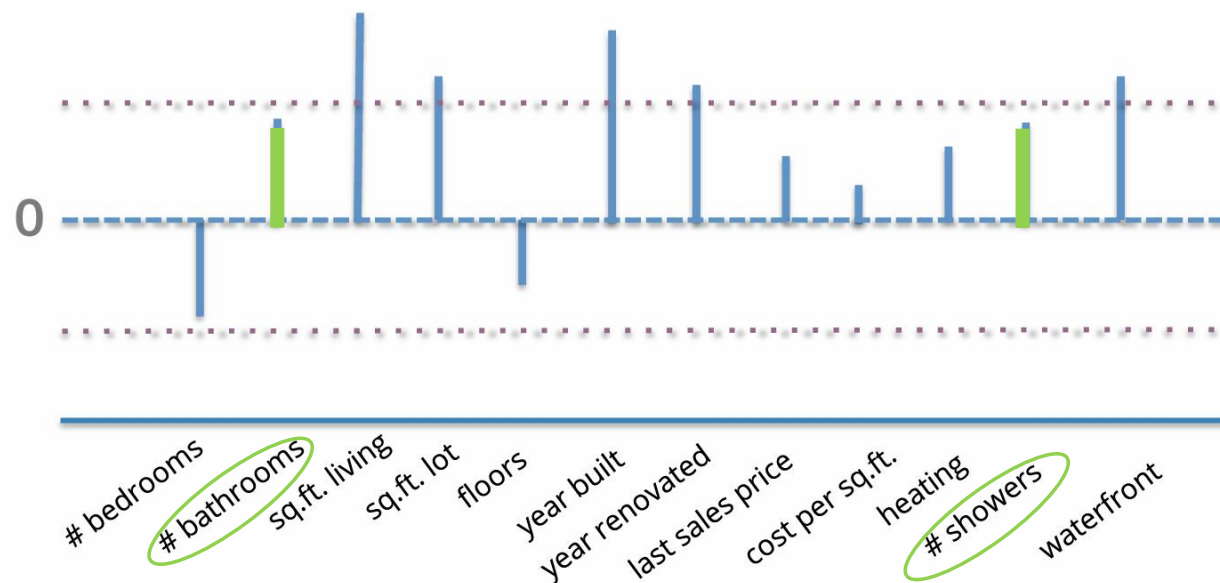
阈值化岭回归系数?

- **P1:** 如果人为设定一个阈值，把小于阈值的系数直接置为0，确实可以做特征选择，但结果高度依赖阈值，不稳定。



阈值化岭回归系数？

- **P2**: 如果考虑两个高度相关的特征（#bathrooms and #showers），在岭回归下，它们的系数通常都会被同时收缩，而不会只保留一个。



模型最终没有选择任何与浴室相关的特征！

Lasso回归: L1正则化回归

Total cost = measure of fit + measure of magnitude of coefficients

$\text{RSS}(\mathbf{w})$

$$||\mathbf{w}||_1 = |w_0| + \cdots + |w_D|$$

Leads to **sparse** solutions!

Lasso 回归
(a.k.a. L1 regularization, L1正则化)

Lasso回归：L1正则化回归

如果 $\hat{\mathbf{w}}$ 是通过最小化以下函数得到的： $\text{RSS}(\mathbf{w}) + \lambda \|\mathbf{w}\|_1$

λ : 调节“拟合数据”和“特征稀疏度”之间的权衡。

- 如果 $\lambda=0$:

退化为普通最小二乘惩罚，解为 $\hat{\mathbf{w}}^{\text{LS}}$

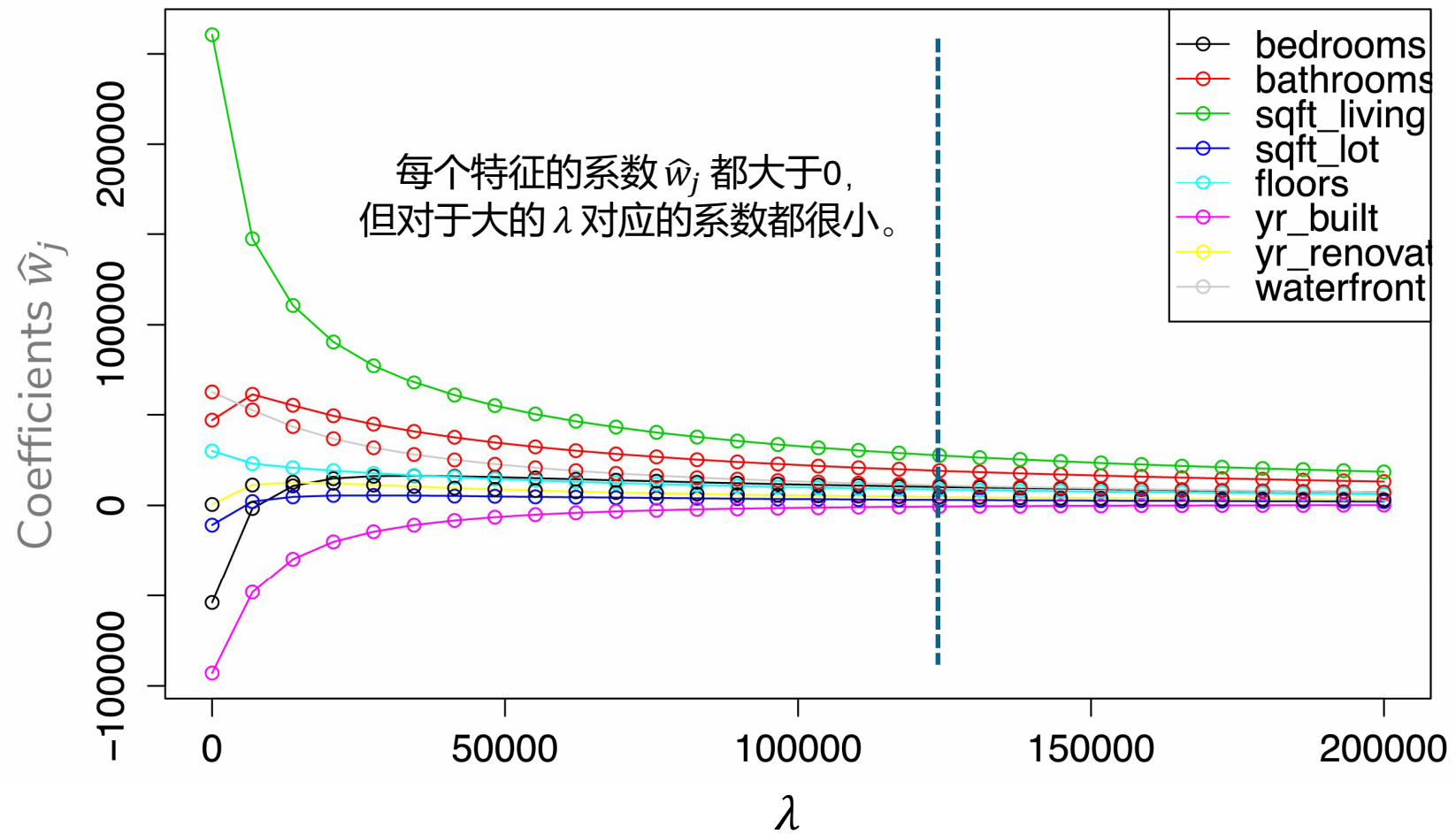
- 如果 $\lambda=\infty$:

当 $\mathbf{w} \neq \mathbf{0}$ ，代价都是 ∞ ；只有当 $\mathbf{w} = \mathbf{0}$ 时，代价 = $\text{RSS}(\mathbf{0})$

- 如果 λ 取中间值:

解介于两者之间，满足 $0 \leq \|\hat{\mathbf{w}}\|_1 \leq \|\hat{\mathbf{w}}^{\text{LS}}\|_1$

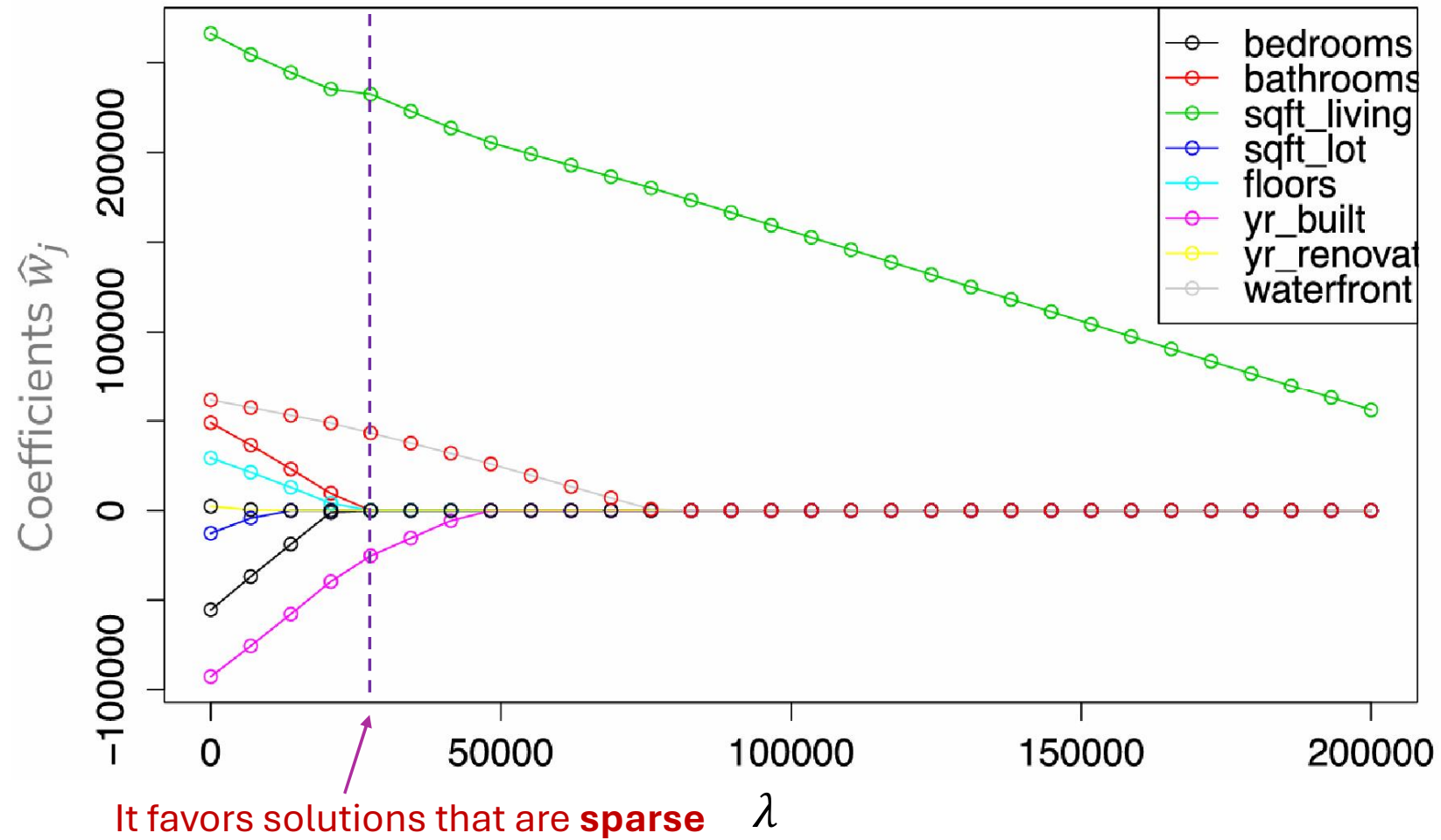
系数路径 - 岭回归



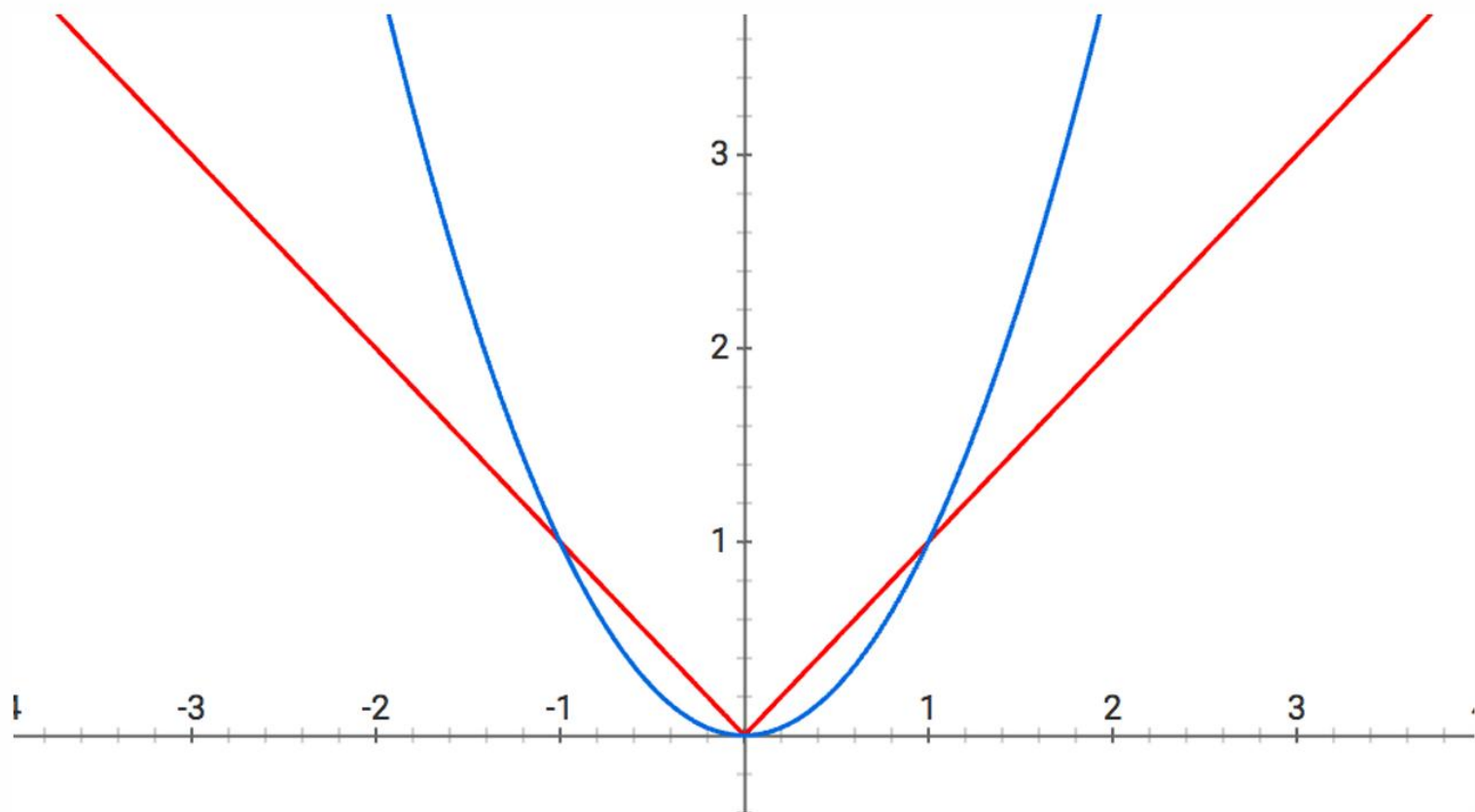
重看多项式拟合示例

- 如果我们重新拟合同一个高阶多项式，但这次使用 **Lasso 回归**，会发生什么？
- 我们将考察几个不同的 λ 取值.....

系数路径 – Lasso回归



和范数的形状有关



当 w_j 较小时, w_j^2 会非常小!

提纲

- 模型拟合与修正
- 岭回归模型
- **Lasso 回归模型**
 - 特征选择任务
 - 模型拟合
 - 正则超参选择
 - 解稀疏性几何直观
- 网络正则化

回顾：如何优化目标？

- 为了求解 $\hat{\mathbf{w}}$ ，我们通常会对总代价函数取梯度，然后：
 - 推导出闭式解 (closed-form solution) ；
 - 或在梯度下降 (gradient descent) 算法中使用该梯度进行迭代更新。

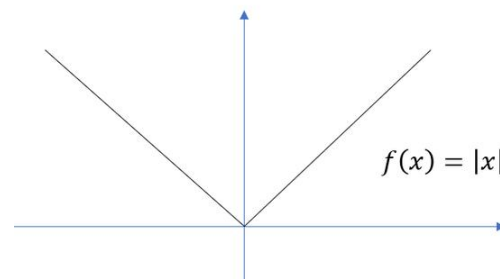
优化 Lasso 回归

- Lasso 总损失函数: $RSS(\mathbf{w}) + \lambda \|\mathbf{w}\|_1$

- 问题:

- $|\mathbf{w}_j|$ 的导数是什么?

- $\mathbf{w}_j > 0$ 时, 导数为+1; $\mathbf{w}_j < 0$ 时, 导数为-1;
 - 但 $\mathbf{w}_j = 0$ 时导数不可导 (需要用次梯度 sub-gradient)



$$f(x) = |x|, \text{ 则 } g = \begin{cases} \text{sgn}(x) & x \neq 0 \\ \text{any } \in [-1, 1] & x = 0 \end{cases}$$

- 即使能计算导数, 也没有闭式解

- 所以 Lasso 通常用迭代优化算法 (如坐标下降、次梯度法、近端梯度下降、交替方向乘子法、最小角回归法等) 来求解。

方法：坐标下降

- 总损失目标：最小化某个函数 g

$$g(\mathbf{w}) = g(w_0, w_1, \dots, w_D)$$

- 通常：同时所有坐标上找到最小值很难，但在单个坐标上优化却很容易
- 坐标下降 (Coordinate Descent) 算法流程：

初始化 $\hat{\mathbf{w}} = 0$ ：

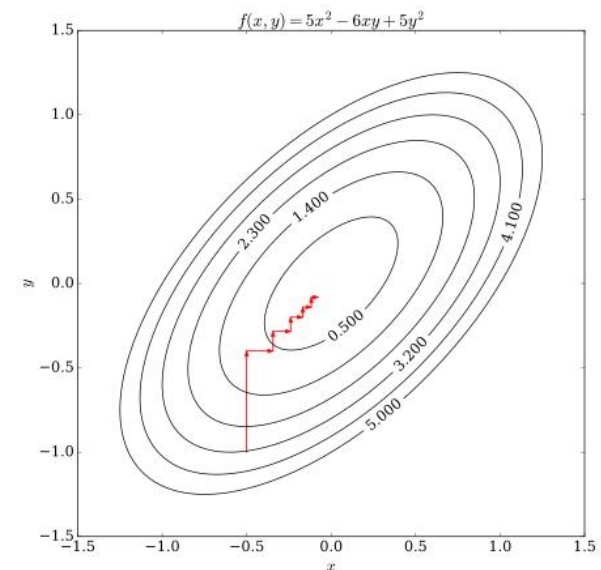
当尚未收敛时，重复：

 选择一个坐标 j ；

 更新该坐标 $\hat{w}_j \leftarrow \min g(\mathbf{w})$

坐标下降法分析

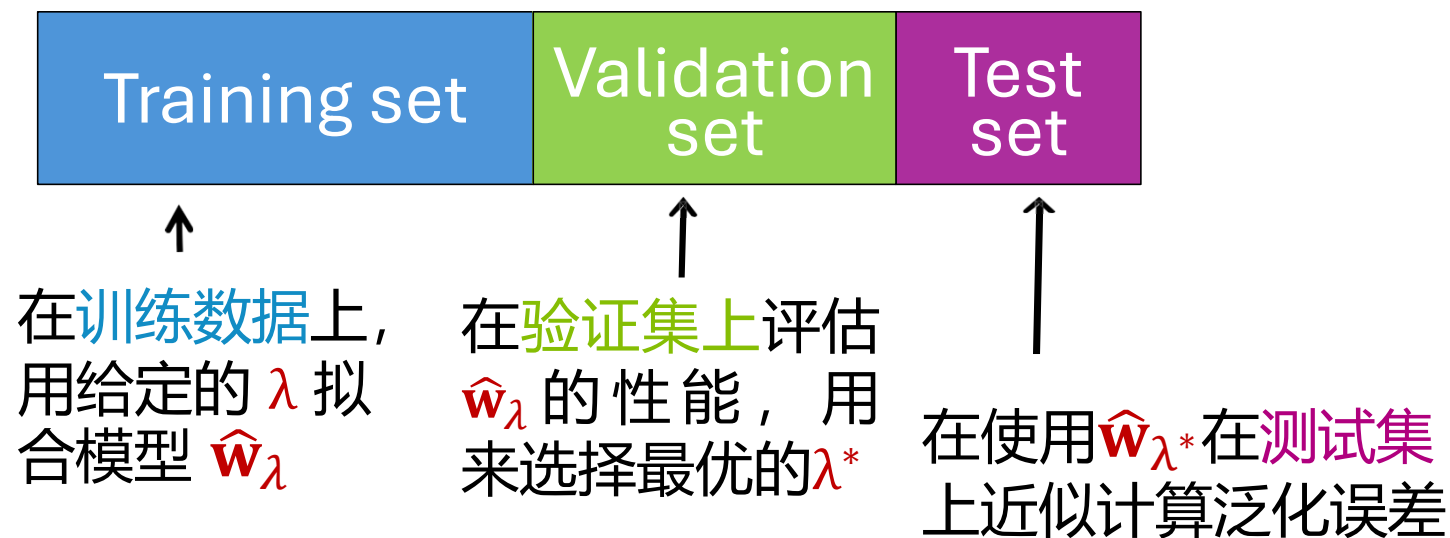
- 如何选取下一个更新坐标?
 - At random (random or stochastic coordinate descent), round robin, ...
- 优势
 - 无需选择步长
 - 在一定条件下收敛，如强凸性
 - 对Lasso目标函数收敛
- 何时停止?
 - 对于凸问题，采取越来越小的步长
 - 测量所有特征在一个完整循环中步长的大小
 - 直到最大步数达到阈值



提纲

- 模型拟合与修正
- 岭回归模型
- **Lasso 回归模型**
 - 特征选择任务
 - 模型拟合
 - 正则超参选择
 - 解稀疏性几何直观
- 神经网络正则化

如果数据量足够

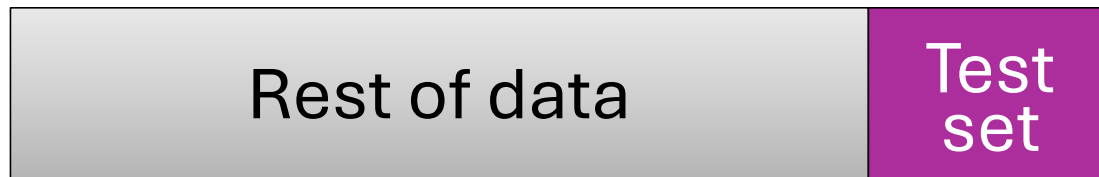


先从一个较小的数据集开始



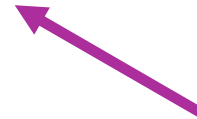
All data

生成测试集并保留下来



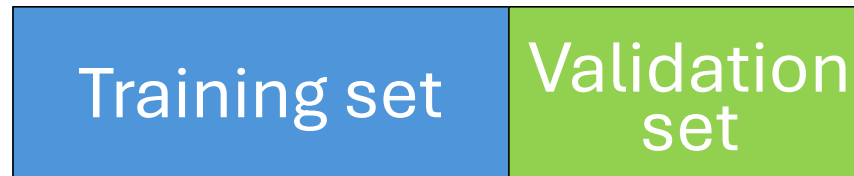
如何利用剩余数据?

Rest of data



同时用于训练和验证,
但不能太简单粗暴

回想一下最简单的方法



Small validation set

- 验证集是否足够用来比较不同 λ 值下 $\hat{\mathbf{w}}_\lambda$ 的性能?

No

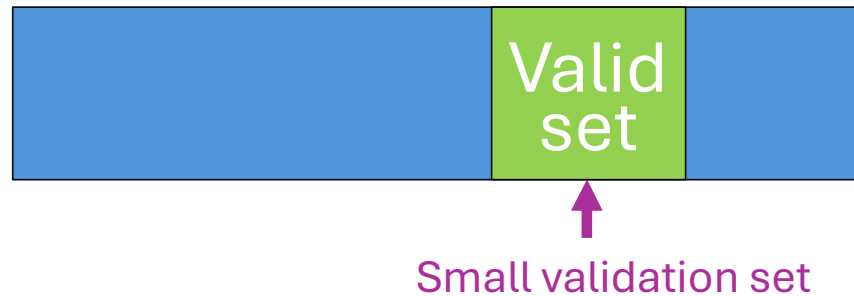
选择验证集



Small validation set

- 并不一定要用最后几条数据来构造验证集。
- 实际上，可以使用数据中的任意子集。

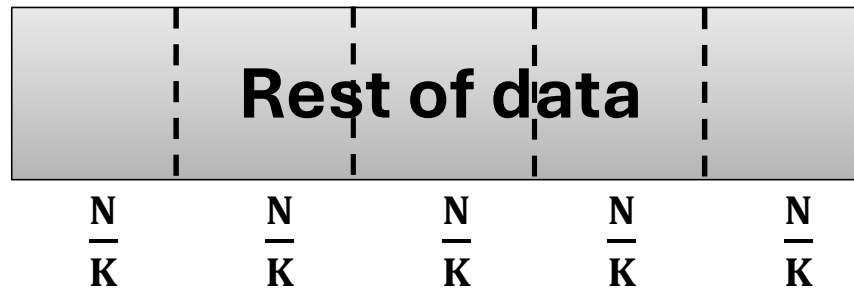
选择验证集



- 该用哪个子集呢?
- 答案是：对所有子集选择取平均性能。

ALL!

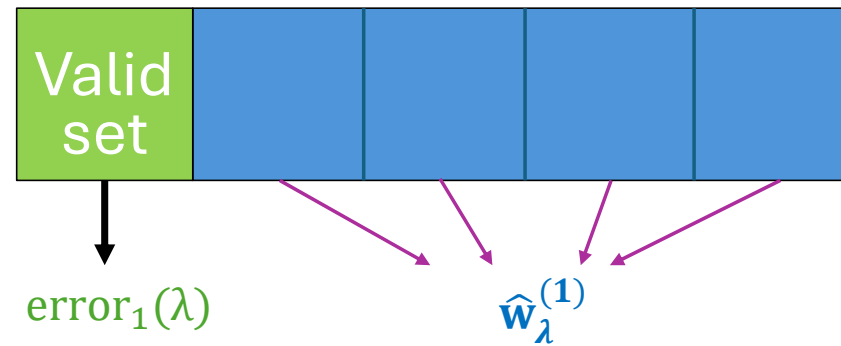
K折交叉验证



- 将数据随机划分为 K 个大小相近的子集 (folds)

在其他所有步骤用同样的划分

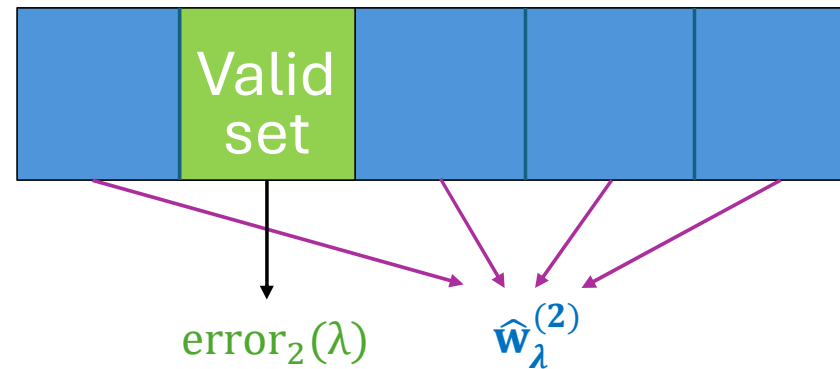
K折交叉验证



- 对于 $k = 1, \dots, K$:

- 每次选择其中 1 个子集作为验证集，其余 $K-1$ 个子集作为训练集；在训练块上估计 $\hat{w}_\lambda^{(k)}$ ；
- 重复 K 次，使得每个子集都恰好被用作一次验证集，计算误差 $error_k(\lambda)$ 。

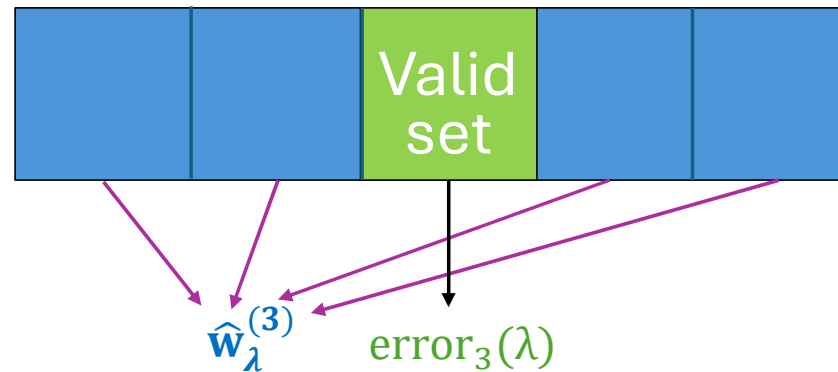
K折交叉验证



- 对于 $k = 1, \dots, K$:

- 每次选择其中 1 个子集作为验证集，其余 $K-1$ 个子集作为训练集；在训练块上估计 $\hat{w}_\lambda^{(k)}$ ；
- 重复 K 次，使得每个子集都恰好被用作一次验证集，计算误差 $\text{error}_k(\lambda)$ 。

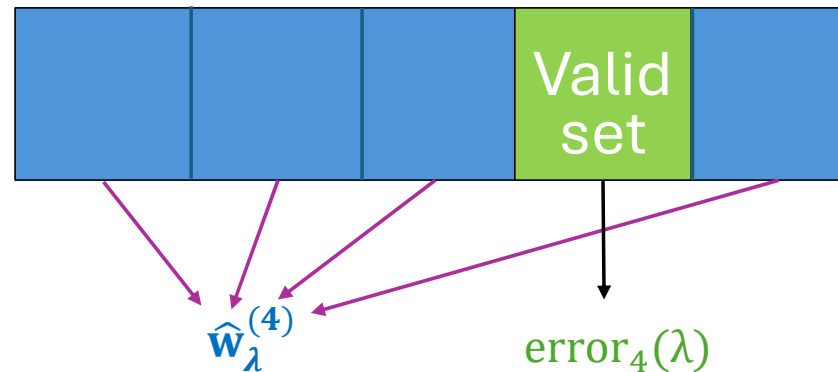
K折交叉验证



- 对于 $k = 1, \dots, K$:

- 每次选择其中 1 个子集作为验证集，其余 $K-1$ 个子集作为训练集；在训练块上估计 $\hat{w}_\lambda^{(k)}$ ；
- 重复 K 次，使得每个子集都恰好被用作一次验证集，计算误差 $\text{error}_k(\lambda)$ 。

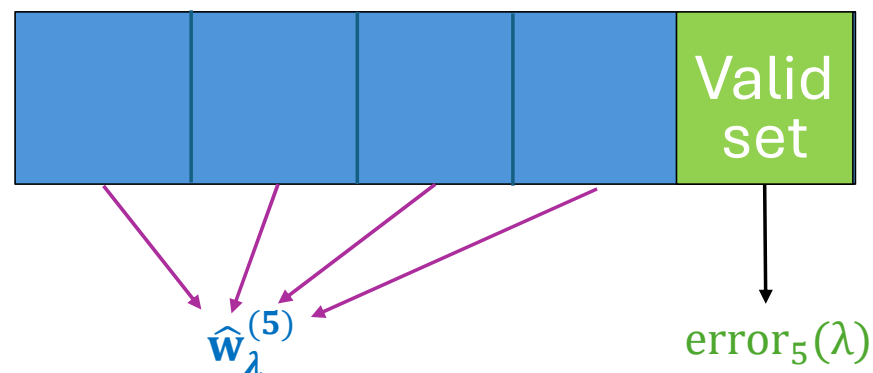
K折交叉验证



- 对于 $k = 1, \dots, K$:

- 每次选择其中 1 个子集作为验证集，其余 $K-1$ 个子集作为训练集；在训练块上估计 $\hat{w}_\lambda^{(k)}$ ；
- 重复 K 次，使得每个子集都恰好被用作一次验证集，计算误差 $error_k(\lambda)$ 。

K折交叉验证



- 对于 $k = 1, \dots, K$:
 - 每次选择其中 1 个子集作为验证集，其余 $K-1$ 个子集作为训练集；在训练块上估计 $\hat{w}_\lambda^{(k)}$ ；
 - 重复 K 次，使得每个子集都恰好被用作一次验证集，计算误差 $\text{error}_k(\lambda)$ 。

K折交叉验证



- 对每个 λ 的选择重复上述过程

Choose λ^* to minimize $CV(\lambda)$

如何决定K值

- 从形式上看：当验证集大小为 1（即 $K=N$ ，留一交叉验证）时，近似最好。
- 但计算量巨大，每个 λ 都需要训练 N 个模型。

Leave-one-out
cross validation

- 实际应用：通常取 $K=5$ 或 $K=10$ ，在近似精度和计算成本之间达到平衡。

5-fold CV

10-fold CV

通过交叉验证为Lasso选择 λ

- 交叉验证的作用是选择能带来最佳预测精度的 λ 。
- 它往往会偏向于更不稀疏的解，因此选择的 λ 通常比用于特征选择的最佳值要小。

参考文献：Murphy, Probabilistic Machine Learning: An Introduction, 2022深入讨论

Lasso回归的问题

- **Lasso**

- 当一组特征高度相关时，Lasso 往往会在它们中任意选择其中一部分。
- 但在很多情况下，我们更希望它们能被一起选择。

- **岭回归**

- 经验上，岭回归在预测性能上往往优于 Lasso；
- 但 Lasso 会得到更加稀疏的解（更利于特征选择）。

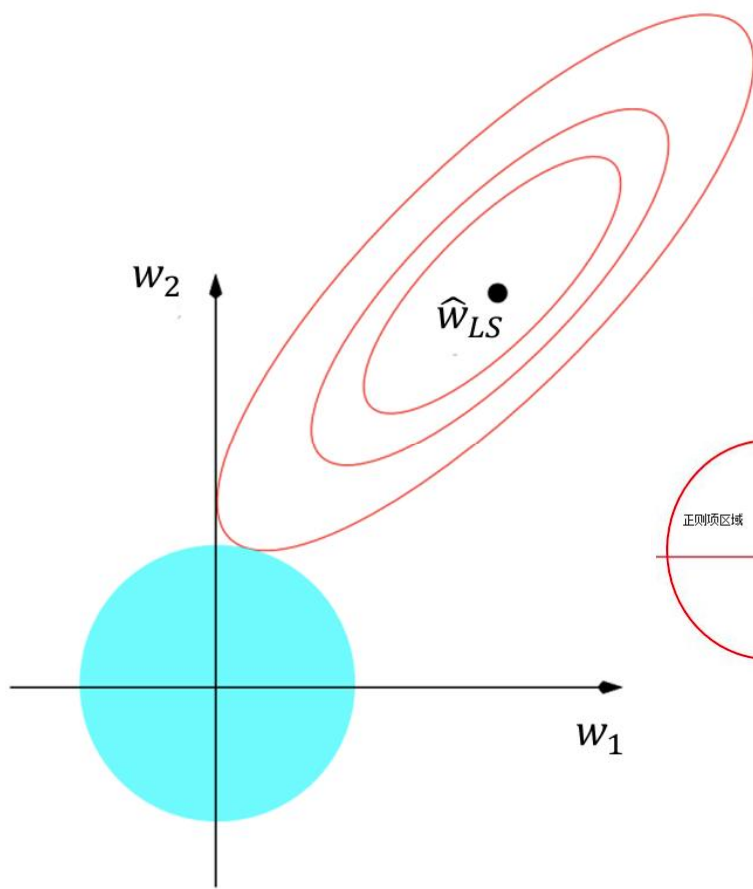
- **弹性网络 (Elastic Net)**

- 结合了 Lasso 与 岭回归 的优点；
- 使用同时包含 L1 与 L2 的惩罚项。

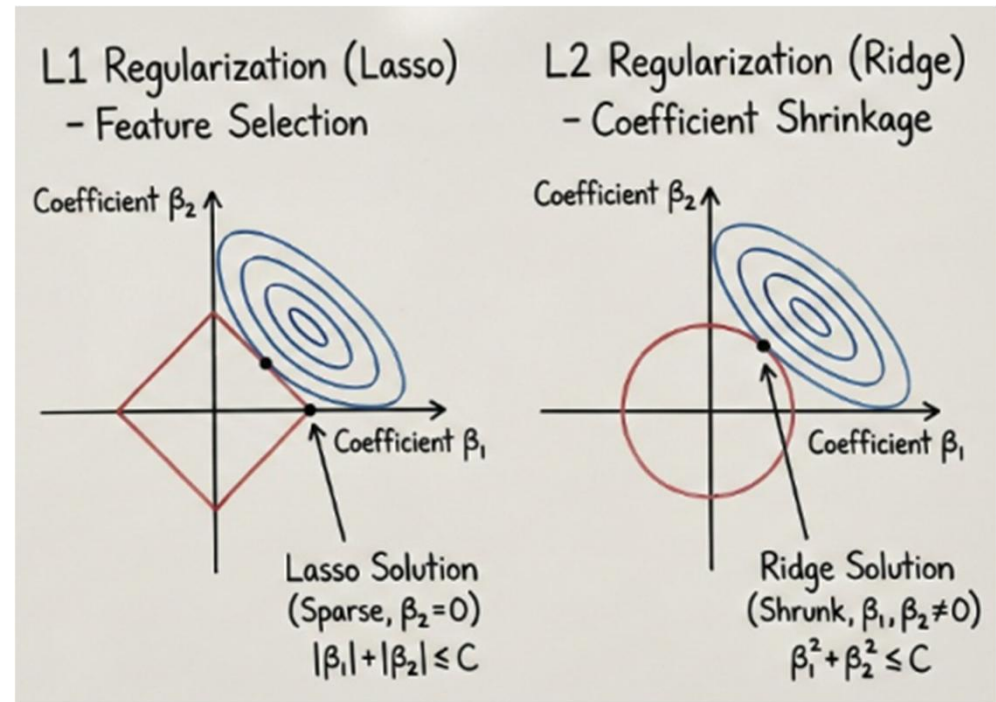
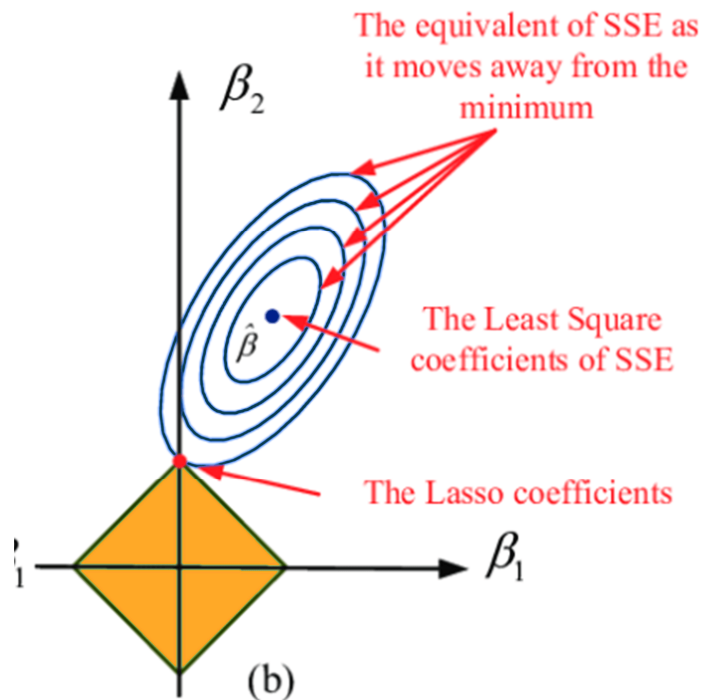
$$\mathbf{w} \sim \min_{\mathbf{w}} \text{RSS}(\mathbf{w}) + \lambda_1 \|\mathbf{w}\|_1 + \lambda_2 \|\mathbf{w}\|_2^2$$

提纲

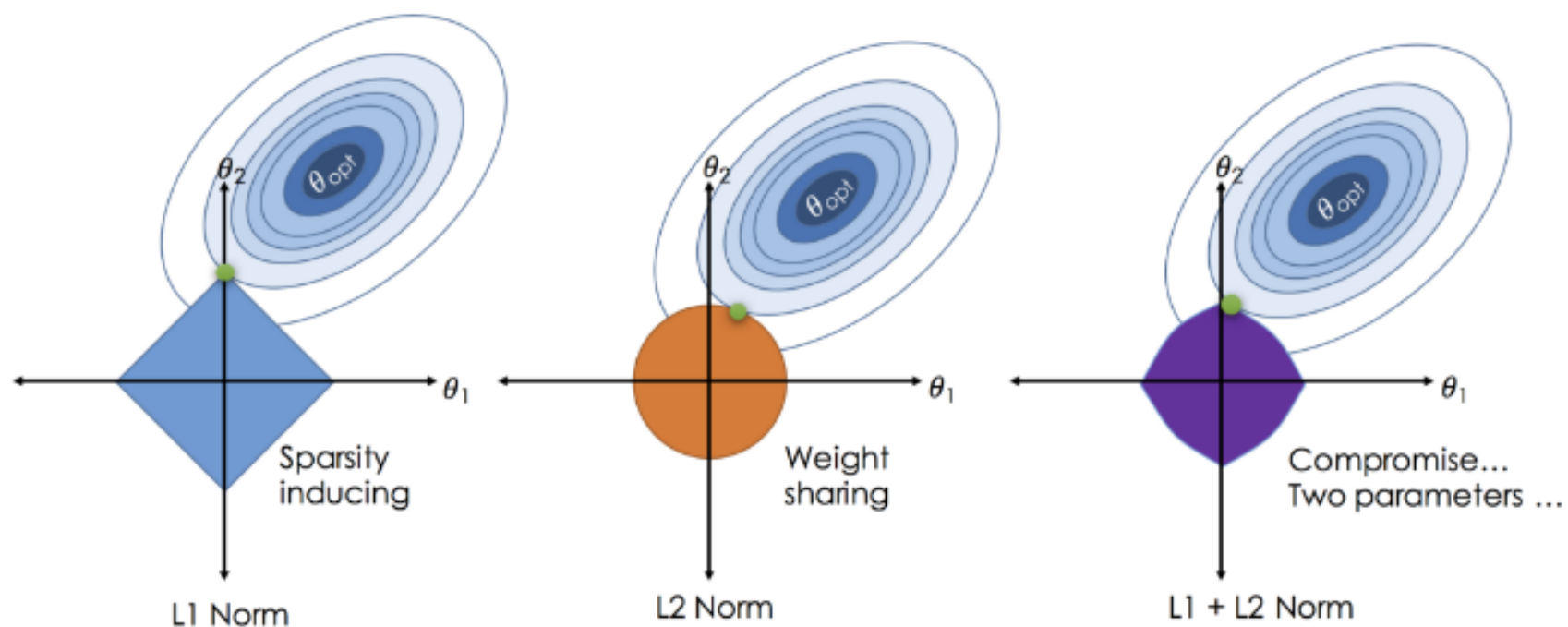
- 模型拟合与修正
- 岭回归模型
- **Lasso 回归模型**
 - 特征选择任务
 - 模型拟合
 - 正则超参选择
 - 解稀疏性几何直观
- 神经网络正则化



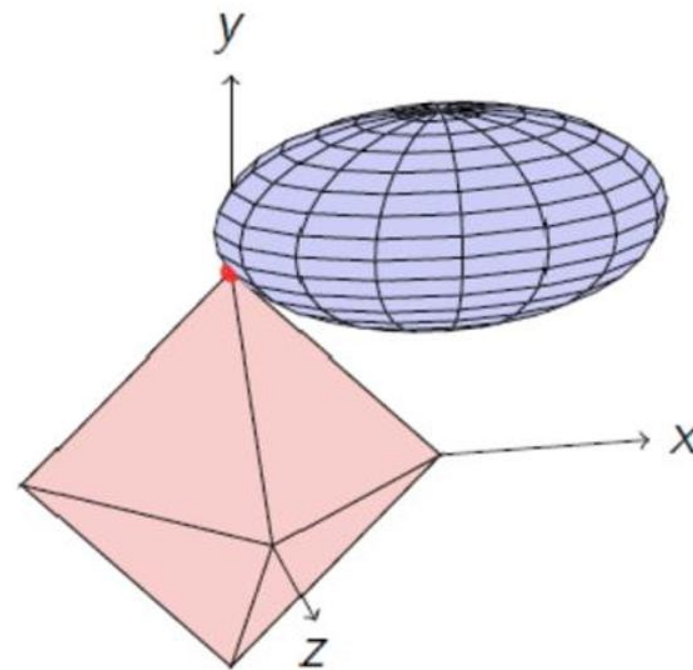
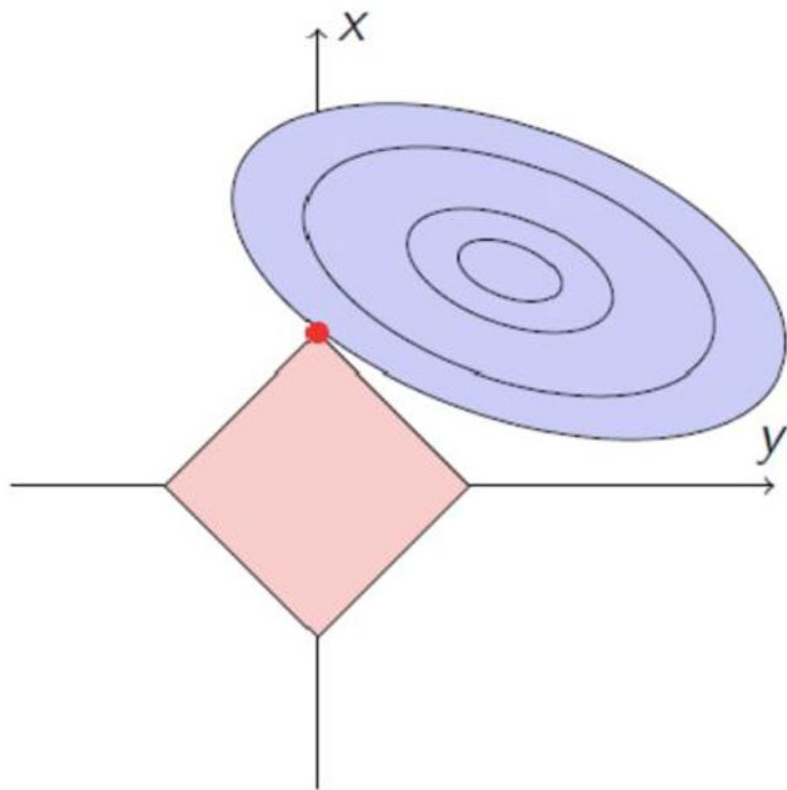
几何理解：损失可视化



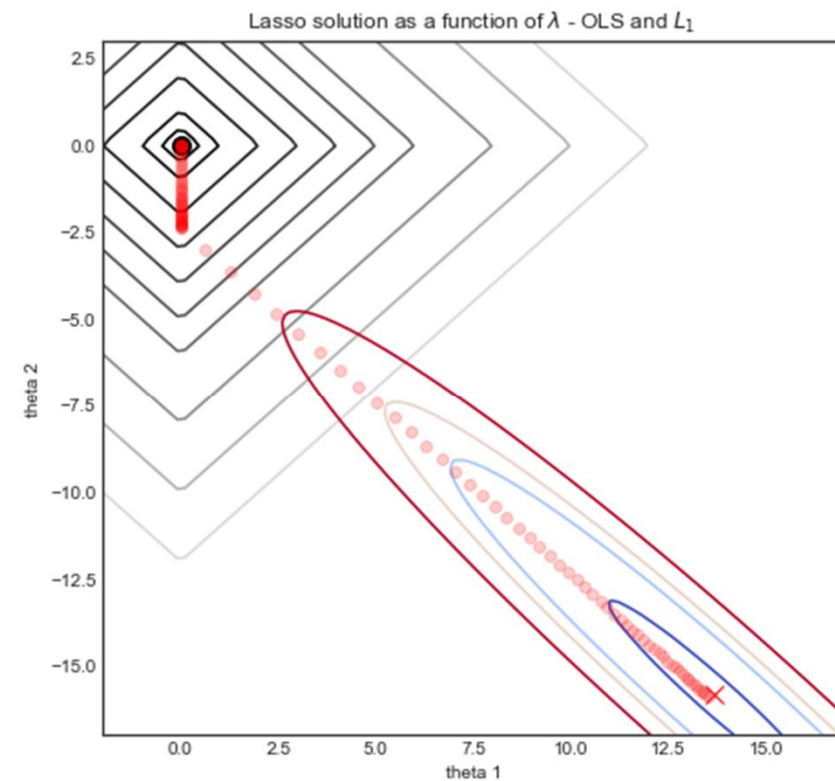
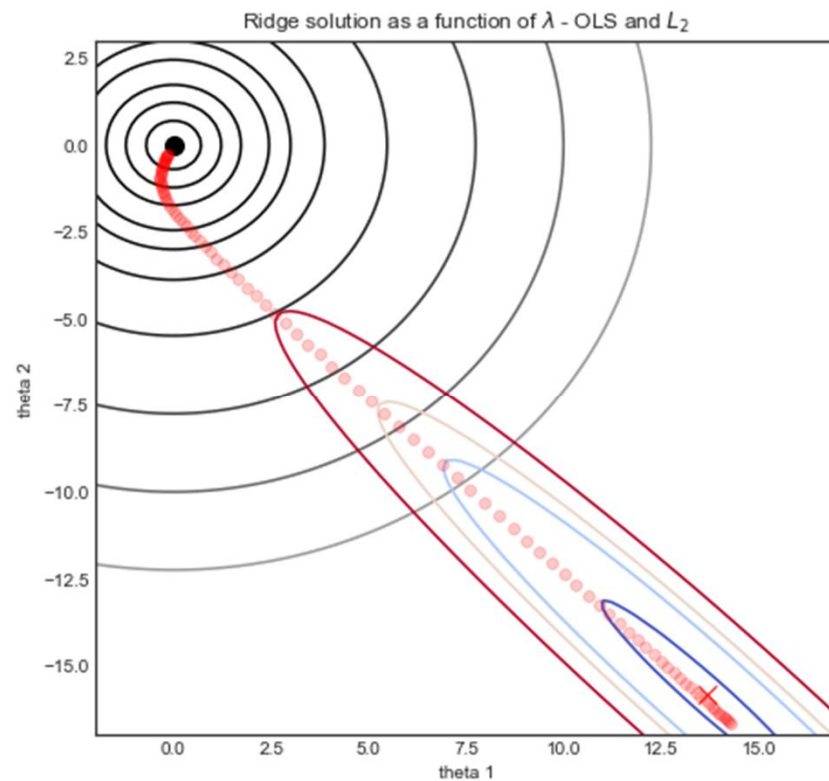
几何理解：损失可视化



几何理解：损失可视化



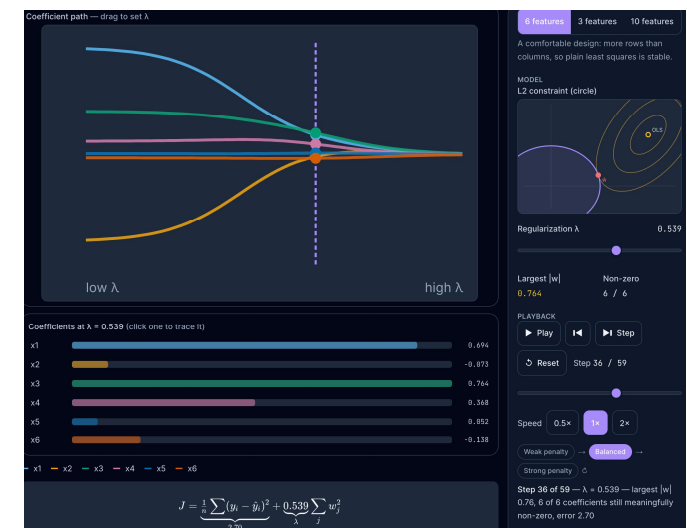
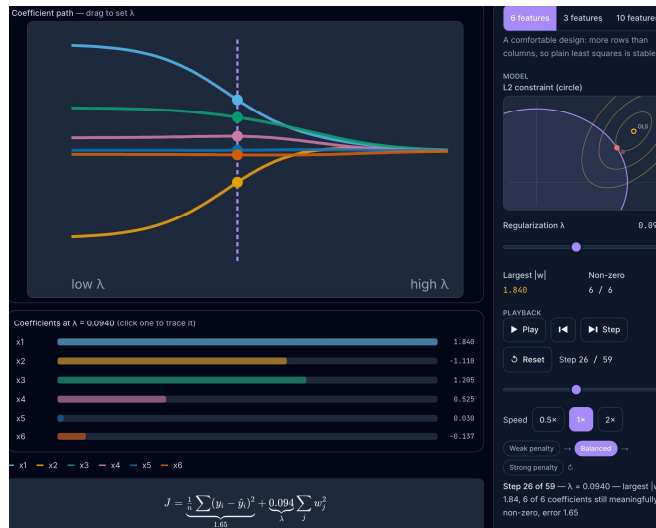
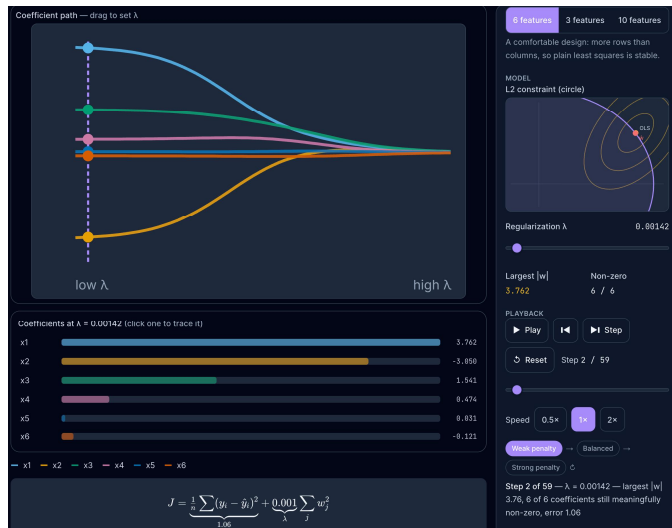
岭回归 vs. Lasso回归优化



优化可视化：岭回归

- 交互式可视化展示

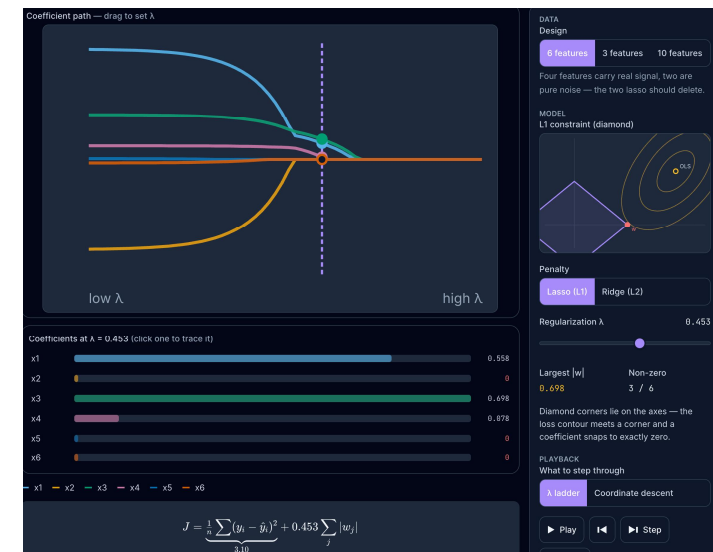
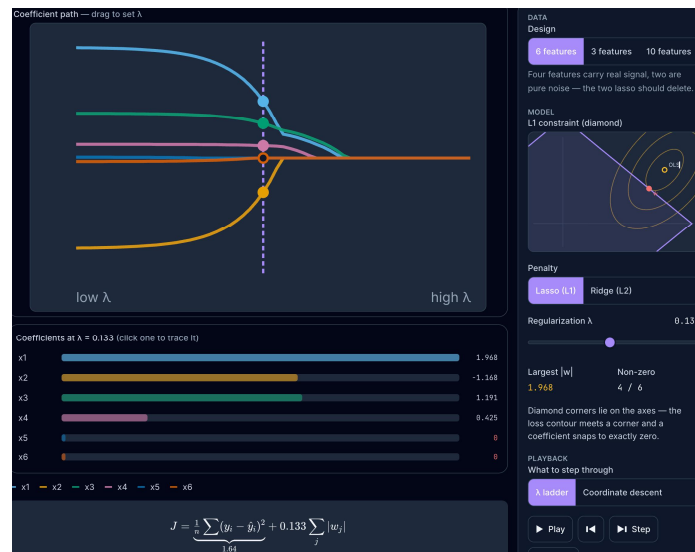
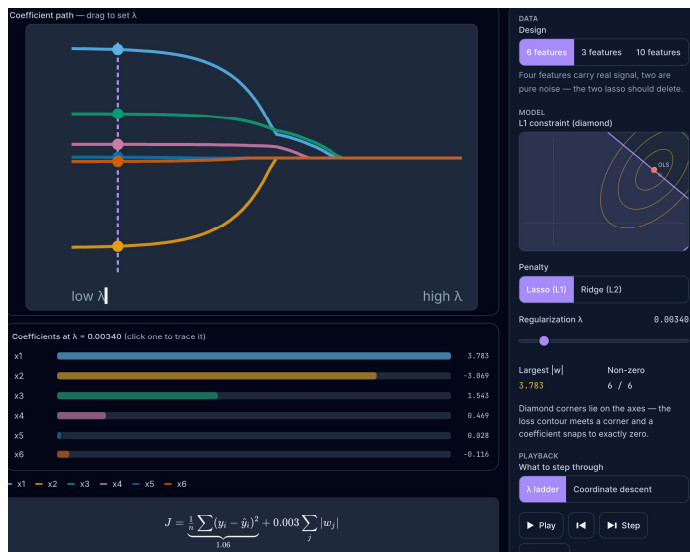
- <https://www.mlvisualization.com/visualizations/ridge-regression>



优化可视化: Lasso回归

- 交互式可视化展示

- <https://www.mlvisualization.com/visualizations/ridge-regression>
- <https://animint-manual-en.netlify.app/ch11/>



提纲

- 模型拟合与修正
- 岭回归模型
- **Lasso** 回归模型
- **神经网络正则化**

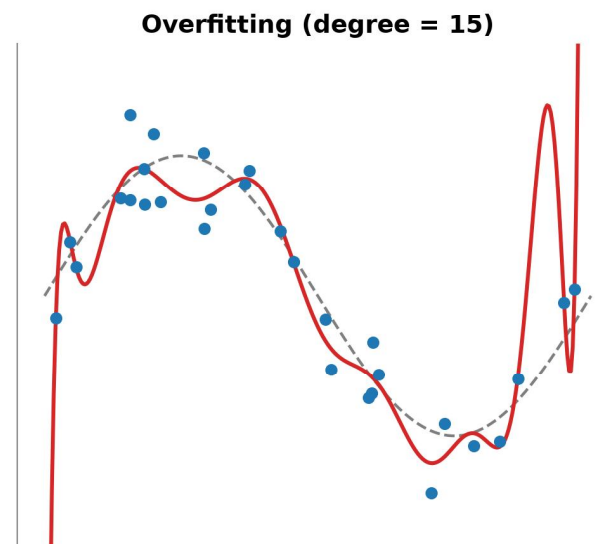
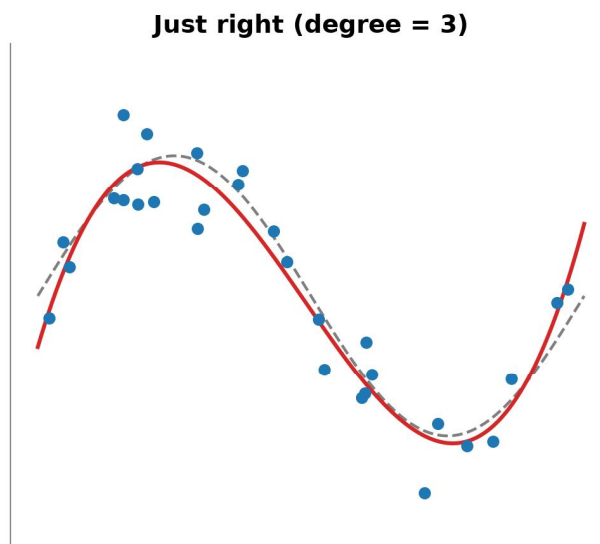
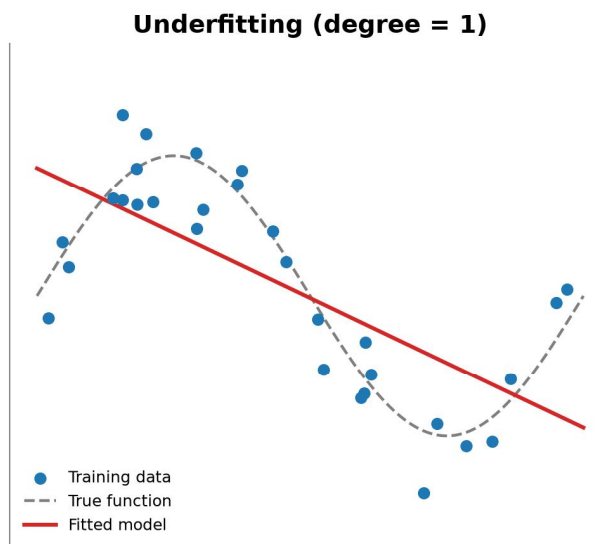
考虑泛化性

- 神经网络

- 过度参数化
- 拟合能力强

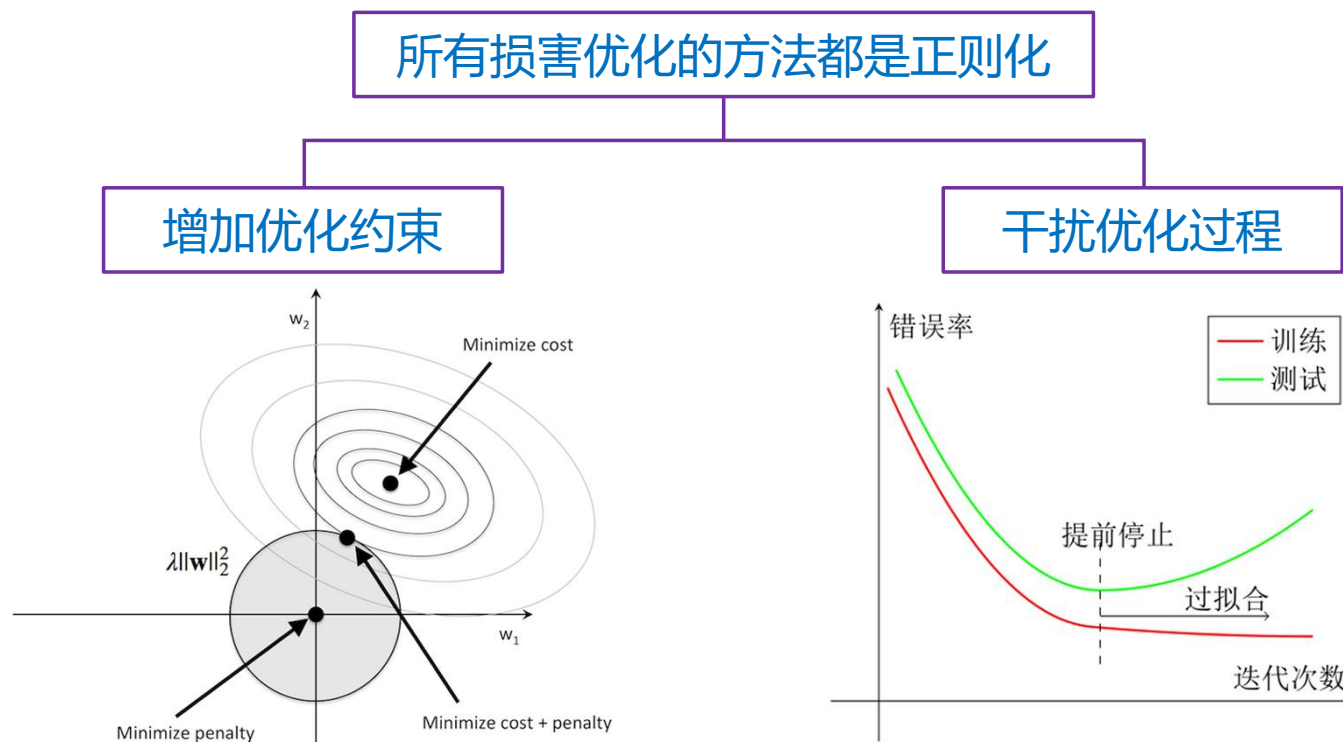


泛化性差



正则化

- **正则化**是一类通过限制模型复杂度，从而避免过拟合，提高泛化能力的方法，比如**引入约束**、**增加先验**、**提前停止**等。

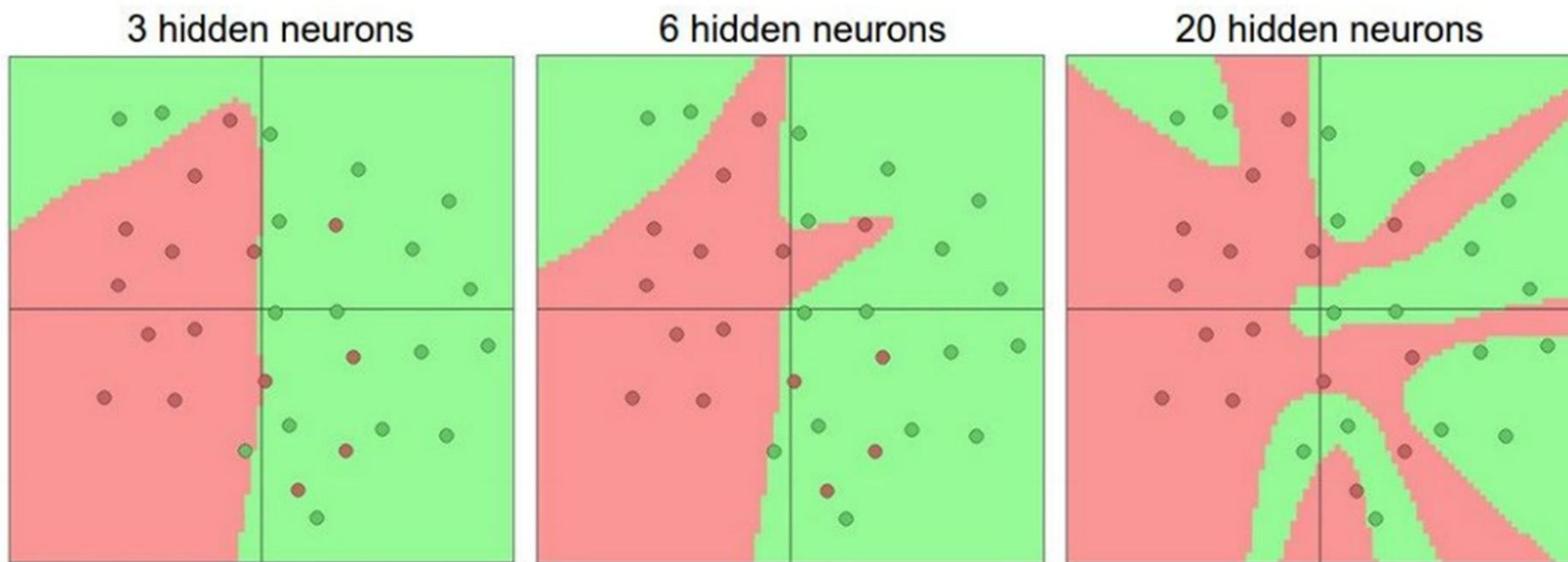


如何提高泛化能力

- L1 和 L2 正则化
- 权重衰减
- 提前停止
- 随机丢弃
- 数据增强
- 标签平滑
-

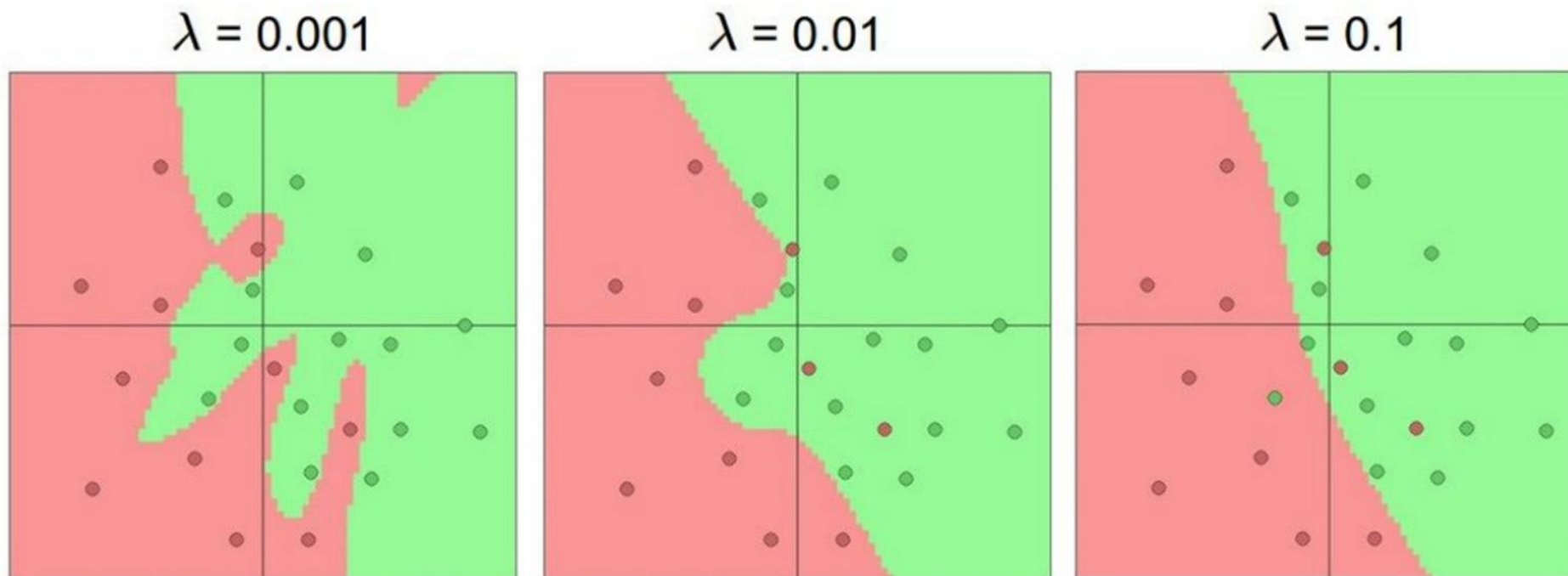
神经网络示例

- 隐藏层的不同神经元个数



神经网络示例

- 不同的正则化系数



权重衰减 (Weight Decay)

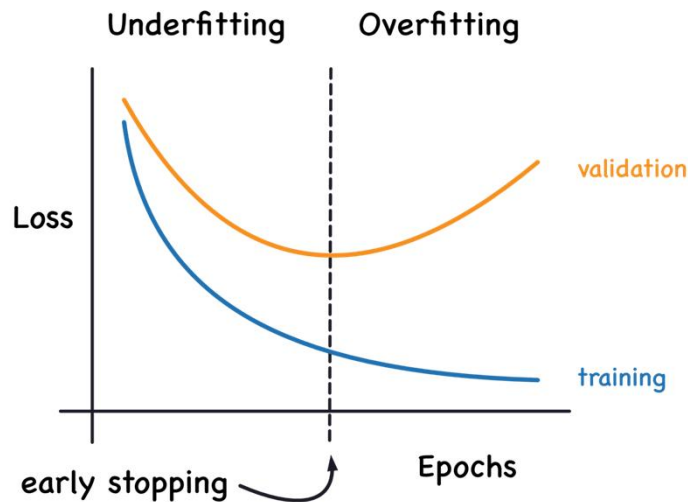
- 每次参数更新时，引入一个衰减系数

$$\theta_t \leftarrow (1 - \beta)\theta_{t-1} - \eta \nabla_t$$

- ∇_t 为第t步的更新梯度， η 为学习率， β 为**权重衰减系数**
- 标准的随机梯度下降中，权重衰减和 L2 正则化的效果相同。
- 较为复杂的优化方法（比如 Adam）中，权重衰减和 L2 正则化并不等价。

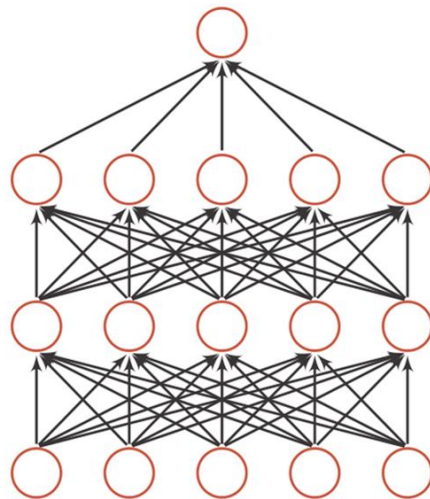
提前停止 (Early stopping)

- 神经网络拟合能力强，易过拟合。训练时可以使用验证集上的误差代替期望误差。
- 使用一个验证集 (Validation Dataset) 来测试每一次迭代的参数在验证集上是否最优。如果在验证集上的指标 (Loss, 错误率等) 较长时间内不再下降，就停止迭代。

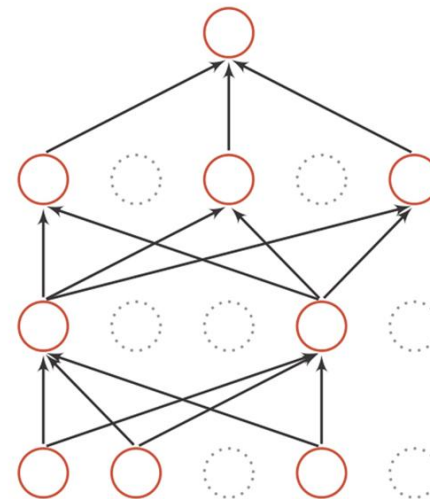


随机丢弃 (Dropout)

- 训练一个网络时，随机丢弃一部分神经元来避免过拟合
- 可看作是一种集成学习：每做一次丢弃，相当于从原始的网络中采样得到一个子网络
- 可解释为一种贝叶斯学习的近似



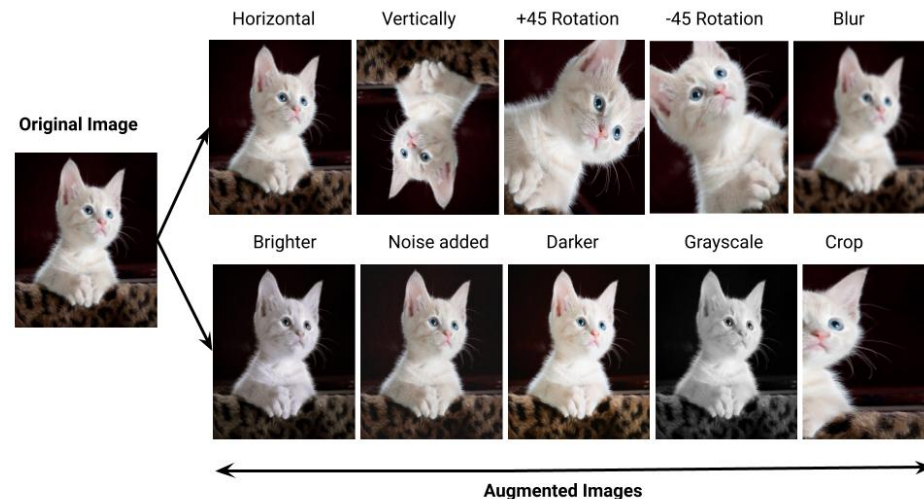
(a) 标准网络



(b) Dropout 后的网络

数据增强

- 通过算法对数据进行转变，引入噪声等方法来增加数据的多样性，以此**提高模型鲁棒性**，避免过拟合。
- 例如，图像数据的增强方法：
 - 旋转 (Rotation)
 - 翻转 (Flip)
 - 缩放 (Zoom in/out)
 - 平移 (Shift)
 - 加噪声 (Noise)
 -



标签平滑 (Label Smoothing)

- 在输出标签中添加噪声来避免模型过拟合
 - 一个样本x的标签一般用 one-hot 向量表示

$$y = [0, \dots, 0, 1, 0, \dots, 0]^T \quad \text{硬目标 (Hard Target)}$$

- 引入一个噪声对标签进行平滑，即假设样本以 ϵ 的概率为其 它类。

平滑后的标签为

$$y = \left[\frac{\epsilon}{K-1}, \dots, \frac{\epsilon}{K-1}, 1 - \epsilon, \frac{\epsilon}{K-1}, \dots, \frac{\epsilon}{K-1} \right]^T \quad \text{软目标 (Soft Target)}$$

- 标签平滑可以避免模型输出过拟合至硬目标上，并且通常不会损害其分类能力

Questions

